ASESII 24A02 Project-Based Quiz 1

Ridge, Lasso, and Regularization for Neural Features

Group 04
Nguyen Thi Yen Nhi (24125071), Nguyen Khang Thinh (24125104)
Nguyen Van Tinh (24125106), Nguyen Le Quoc Huy (24125100)

2026-07-17

Contents

Authorship and contribution statement

We confirm that every group member can explain the submitted analysis. AI use is reported in `Group04_AI_Log.csv`, and the group checked every AI-assisted item used in this report.

Student	ID	Main contributions	Verification performed
Nguyen Thi Yen Nhi	24125071	[Work]	[Check]
Nguyen Khang Thinh	24125104	[Work]	[Check]
Nguyen Van Tinh	24125106	[Work]	[Check]
Nguyen Le Quoc Huy	24125100	[Work]	[Check]

Abstract

Maximum 150 words: prediction problem, methods, main evidence, and recommendation.

Reproducible setup

```
required_packages <- c("tidyverse", "glmnet", "broom", "knitr", "car")
missing_packages <- required_packages[
  !vapply(required_packages, requireNamespace, logical(1), quietly = TRUE)
]
if (length(missing_packages)) {
  stop("Install the required packages before rendering: ",
       paste(missing_packages, collapse = ", "))
}

library(tidyverse)
library(glmnet)
library(broom)
library(knitr)
library(car)

# `params` exists only while knitting; the fallback
# keeps interactive chunk runs working.
has_params <- exists("params") && !is.null(params$group_number)
group_number <- if (has_params) as.integer(params$group_number) else 4L
if (length(group_number) != 1L || is.na(group_number) || group_number < 1L) {
  stop("Set params$group_number to your assigned positive group number.")
}

seeds <- list(
  split = 240200L + group_number,
  cv = 240300L + group_number,
  features = 240400L + group_number
)
```

Shared helper functions

```
find_exam_data <- function(filename) {
  input <- knitr::current_input(dir = TRUE)
  input_dir <- if (length(input) && nzchar(input)) dirname(input) else getwd()
  candidates <- c(
    file.path(input_dir, "data", filename),
    file.path(input_dir, "..", "data", filename),
    file.path(getwd(), "data", filename)
  )
  existing <- candidates[file.exists(candidates)]
  if (!length(existing)) stop("Cannot locate ", filename, " by a relative path.")
  hits <- unique(normalizePath(existing, winslash = "/", mustWork = TRUE))
  if (length(hits) > 1L && length(unique(unname(tools::md5sum(hits)))) > 1L) {
```

```
    stop("Multiple nonidentical copies of ", filename, " were found.")
  }
  hits[[1L]]
}

split_rows <- function(n, train_prop = 0.80, seed) {
  stopifnot(n >= 10L, train_prop > 0, train_prop < 1)
  set.seed(seed)
  train <- sort(sample.int(n, floor(train_prop * n), replace = FALSE))
  list(train = train, test = setdiff(seq_len(n), train))
}

fit_scaler <- function(x, tol = 1e-12) {
  x <- as.matrix(x)
  center <- colMeans(x)
  centered <- sweep(x, 2L, center, "-")
  scale <- sqrt(colMeans(centered^2))
  keep <- is.finite(scale) & scale > tol
  if (!any(keep)) stop("No nonconstant training predictor remains.")
  list(
    center = center[keep],
    scale = scale[keep],
    keep = keep,
    dropped = colnames(x)[!keep]
  )
}

apply_scaler <- function(x, prep) {
  x <- as.matrix(x)[, prep$keep, drop = FALSE]
  x <- sweep(x, 2L, prep$center, "-")
  sweep(x, 2L, prep$scale, "/")
}

make_foldid <- function(n, k = 5L, seed) {
  if (k < 3L || k > n) stop("Require 3 <= k <= number of training rows.")
  set.seed(seed)
  sample(rep(seq_len(k), length.out = n))
}

fit_cv_glmnet <- function(x, y, alpha, foldid) {
  glmnet::cv.glmnet(
    x = x,
    y = y,
    family = "gaussian",
    alpha = alpha,
    foldid = foldid,
    standardize = FALSE,
    intercept = TRUE,
```

```
    type.measure = "mse"
  )
}

score_regression <- function(y, prediction) {
  y <- as.numeric(y)
  prediction <- as.numeric(prediction)
  if (length(y) != length(prediction) || any(!is.finite(c(y, prediction)))) {
    stop("Metrics require equal-length finite vectors.")
  }
  c(RMSE = sqrt(mean((y - prediction)^2)),
    MAE = mean(abs(y - prediction)))
}

count_nonzero <- function(fit, s = "lambda.min", tol = 1e-8) {
  b <- as.matrix(stats::coef(fit, s = s))
  slopes <- b[rownames(b) != "(Intercept)", 1L]
  sum(abs(slopes) > tol)
}

safe_condition_numbers <- function(x, lambda, tol = 1e-10) {
  eigenvalues <- eigen(
    crossprod(x) / nrow(x),
    symmetric = TRUE,
    only.values = TRUE
  )$values
  largest <- max(eigenvalues)
  smallest <- max(min(eigenvalues), 0)
  cutoff <- tol * max(1, largest)
  c(
    gram = if (smallest <= cutoff) Inf else largest / smallest,
    ridge = (largest + lambda) / (smallest + lambda)
  )
}
```

1 Problem 1. Prediction design and leakage control

1.1 1A. Target and predictors

```
config <- list(
  file = "prostate.csv",
  response = "lpsa",
  excluded = "lpsa"
)
data_raw <- read.csv(find_exam_data(config$file), check.names = FALSE)
```

```r
predictors <- setdiff(names(data_raw), config$excluded)
analysis_data <- data_raw[, c(config$response, predictors), drop = FALSE]

if (anyNA(analysis_data)) stop("Declare a training-only missing-value plan.")
if (!all(vapply(analysis_data, is.numeric, logical(1)))) {
  stop("The assigned response and predictors must be numeric.")
}
```

Assignment. Group 04 is even-numbered, so the assigned dataset is `prostate.csv` and the assigned response is `lpsa`.

Dataset overview. The `prostate.csv` file originates from the study by Stamey et al. (1989), which examined the relationship between prostate-specific antigen (PSA) levels and several clinical and histological measurements in men with clinically localized prostate cancer who underwent radical prostatectomy. The dataset contains 97 observations and 9 variables — one continuous response (`lpsa`) and eight candidate predictors covering tumour characteristics, patient demographics, and biopsy summaries. All variables are numeric and the dataset has no missing values, so no imputation or factor encoding is required. Individual variable definitions are detailed in the overview table below.

Response. `lpsa` is the natural logarithm of prostate-specific antigen (ng/mL), a continuous serum marker measured before surgery. It is modelled on the log scale, so every coefficient is read as a change in log PSA per one standard deviation of a standardized predictor.

Unit of observation and study population. One row is one male patient with clinically localized prostate cancer who underwent radical prostatectomy in the study of Stamey et al. (1989). The analysis file contains 97 complete patients. Any conclusion therefore generalizes only to comparable prostatectomy candidates, not to unscreened men in the general population.

Prediction objective. Predict pre-operative `lpsa` from routinely available clinical and biopsy measurements, and compare how OLS, ridge, and lasso trade prediction error against coefficient stability and interpretability when several predictors are correlated.

Candidate predictors. All 8 remaining variables are retained as candidate predictors. Every one is numeric, so the template's type guard passes and the design matrix needs no factor encoding. The variables are drawn from the clinical and pathological records described in Stamey et al. (1989) (Table 1, pp. 1077–1078) and are summarized below.

- `lcavol` — log of cancer volume (log cm^3), estimated by ultrasound before surgery. Larger tumours are expected to produce more PSA.
- `lweight` — log of total prostate weight (log g), measured at prostatectomy. Reflects overall prostate size, including both cancerous and non-cancerous tissue.
- `age` — patient age at the time of diagnosis (years).
- `lbph` — log of the amount of benign prostatic hyperplasia (log cm^2). BPH is non-cancerous prostate enlargement and can independently elevate PSA.
- `svi` — seminal vesicle invasion (binary: 1 = present, 0 = absent), indicating whether cancer has spread beyond the prostate into the seminal vesicles. This is a known marker of advanced disease.
- `lcp` — log of capsular penetration (log cm), measuring the extent to which cancer has invaded through the prostate capsule. Together with `svi`, it captures local disease progression.

- `gleason` — combined Gleason score (integer, ranging from 6 to 9 in this sample), the standard histological grading system for prostate cancer aggressiveness assigned by a pathologist from biopsy tissue.
- `pgg45` — percentage of Gleason grades 4 or 5 in the biopsy specimen (percent, 0–100). Higher values indicate a larger proportion of poorly-differentiated, aggressive tumour tissue.

All eight predictors were recorded before or during the prostatectomy procedure and are defined independently of the serum PSA measurement, so none introduces outcome leakage. The overview table below records the unit, data type, number of distinct values, and missing-value count for each variable.

Exclusions. The project brief requires only that `lpsa` itself be removed from the predictor set, and permits a further exclusion only when a statistical reason is documented. We document none, so exactly one variable is excluded: the response. No candidate predictor is computed from `lpsa`: all 8 are measurements taken before surgery (tumour volume, prostate weight, age, benign hyperplasia, seminal vesicle invasion, capsular penetration, and the two Gleason summaries), and none uses PSA information in its definition.

Leakage rationale. A variable computed from the response carries the answer inside the design matrix. Its least-squares coefficient would absorb nearly all explained variance, the training and cross-validation errors of every method would collapse toward zero, and the resulting model would fail on any new patient whose PSA is not yet known — which is precisely the deployment situation. The comparison itself would also become meaningless: OLS, ridge, and lasso would all be fitting the same leaked signal, so differences between them would reflect how each penalty shrinks that one dominant column rather than how each method handles genuine multicollinearity among clinical predictors. Because none of the eight candidate predictors in `prostate.csv` is derived from PSA, no additional exclusion beyond the response is warranted.

```
meaning <- c(
  lcavol  = "Log cancer volume",
  lweight = "Log prostate weight",
  age     = "Age at diagnosis",
  lbph    = "Log BPH amount",
  svi     = "Seminal vesicle invasion",
  lcp     = "Log capsular penetration",
  gleason = "Gleason score",
  pgg45   = "Percent Gleason grade 4/5",
  lpsa    = "Log prostate-specific antigen"
)
unit <- c(
  lcavol = "log cm3", lweight = "log g",   age   = "years",
  lbph   = "log cm2", svi     = "0/1",     lcp   = "log cm",
  gleason = "6 to 9", pgg45   = "percent", lpsa  = "log ng/mL"
)

# A column absent from the lookups would otherwise print as a silent blank cell.
stopifnot(all(names(data_raw) %in% names(meaning)),
          all(names(data_raw) %in% names(unit)))
```

```r
data_overview <- tibble(
  Variable = names(data_raw),
  Role     = ifelse(names(data_raw) == config$response, "Response", "Predictor"),
  Meaning  = unname(meaning[names(data_raw)]),
  Unit     = unname(unit[names(data_raw)]),
  Type     = vapply(data_raw, function(x) class(x)[1L], character(1)),
  Distinct = vapply(data_raw, dplyr::n_distinct, integer(1), na.rm = TRUE),
  Missing  = vapply(data_raw, function(x) sum(is.na(x)), integer(1))
)

kable(
  data_overview,
  caption = paste("Response and candidate predictors, with the integrity checks",
                  "run before the split.")
)
```

Table 1: Response and candidate predictors, with the integrity checks run before the split.

Variable	Role	Meaning	Unit	Type	Distinct	Missing
lcavol	Predictor	Log cancer volume	log cm3	numeric	93	0
lweight	Predictor	Log prostate weight	log g	numeric	88	0
age	Predictor	Age at diagnosis	years	integer	31	0
lbph	Predictor	Log BPH amount	log cm2	numeric	42	0
svi	Predictor	Seminal vesicle invasion	0/1	integer	2	0
lcp	Predictor	Log capsular penetration	log cm	numeric	30	0
gleason	Predictor	Gleason score	6 to 9	integer	4	0
pgg45	Predictor	Percent Gleason grade 4/5	percent	integer	19	0
lpsa	Response	Log prostate-specific antigen	log ng/mL	numeric	85	0

```r
n_duplicate_rows <- sum(duplicated(analysis_data))
```

Reading the table. The file holds 97 patients and 9 columns. Three integrity results clear the way for the split. Every column is numeric, so the type guard above passes. No cell is missing (`Missing` is zero throughout), so no imputation is performed; the missing-value rule is still declared in 1B as a contingency, because a rule invented after seeing the data would be a decision made with holdout information. And 0 rows are duplicated, so no patient is silently counted twice, which would otherwise inflate the apparent sample size and let the same record fall on both sides of the split. (`BPH` abbreviates benign prostatic hyperplasia, the non-cancerous prostate enlargement recorded by `lbph`.)

The `Distinct` column carries the one substantive surprise. Two predictors are numeric but not continuous: `svi` takes only 2 values (a binary indicator of seminal vesicle invasion) and `gleason` only 4 (an ordinal biopsy score from 6 to 9). We keep both on their numeric scale, which is the standard `glmnet` convention, but their coefficients must later be read with that discreteness in

mind rather than as smooth slopes: a "one standard deviation" change in a binary variable is a convenient fiction, not a clinical intervention.

The `Unit` column exposes the second issue. The predictors live on wildly different scales, from `pgg45` measured in percent (0 to 100) to several variables already on a log scale. Penalized regression shrinks coefficients toward zero, so a predictor measured in large units would be punished merely for its units, not for being uninformative. This is precisely why the design matrix is standardized in 1B, using centering and scaling constants computed on the training rows only.

1.2 1B. Split and train-only preprocessing

```
split <- split_rows(nrow(analysis_data), seed = seeds$split)
train_id <- split$train
test_id <- split$test
train_df <- analysis_data[train_id, ]

x_raw <- as.matrix(analysis_data[, predictors, drop = FALSE])
y_raw <- analysis_data[[config$response]]

x_prep <- fit_scaler(x_raw[train_id, , drop = FALSE])
x_train <- apply_scaler(x_raw[train_id, , drop = FALSE], x_prep)
x_test <- apply_scaler(x_raw[test_id, , drop = FALSE], x_prep)
y_train <- y_raw[train_id]
y_test <- y_raw[test_id]


foldid <- make_foldid(nrow(x_train), k = 5L, seed = seeds$cv)
sum(is.na(analysis_data))
```

```
## [1] 0
```

```
save(x_train, y_train, x_test, y_test, analysis_data, train_id, test_id, foldid,
 ↪  file = "data/tmpData.RData")
```

Report seeds, dimensions, scaling, folds, and leakage safeguards.

1.3 1B. Split and Preprocessing Plan

To ensure a fair comparison among OLS, ridge, lasso, and elastic net, all preprocessing and model development steps were performed using only the training data. The holdout test set was reserved exclusively for the final evaluation and was not used during model fitting, model selection, or hyperparameter tuning.

1.3.1 Data split

The observations were randomly partitioned into training and test sets using the custom `split_rows()` function with a training proportion of 80%. For Group 4, the split seed was **240204**, ensuring that the same partition can be reproduced in future analyses.

```
cat("Training observations:", length(train_id), "\n")
```

```
## Training observations: 77
```

```
cat("Test observations:", length(test_id), "\n")
```

```
## Test observations: 20
```

The training data were used for exploratory analysis, preprocessing, model fitting, and cross-validation, while the holdout test data remained untouched until the final evaluation.

1.3.2 Predictor standardization

Since ridge regression, lasso, and elastic net are sensitive to the scale of predictor variables, all predictors were standardized before model fitting. The centering and scaling parameters were estimated **only from the training data**, and these same values were subsequently applied to the test predictors.

Using training-derived scaling parameters prevents information from the holdout observations from influencing the preprocessing stage and ensures that the test set remains an unbiased evaluation dataset.

1.3.3 Five-fold cross-validation

Model tuning was carried out using five-fold cross-validation on the training data only. Fold assignments were generated using the fixed cross-validation seed **240304** and were shared across all regularized models. Reusing identical folds ensures that differences in cross-validation performance are attributable to the models themselves rather than random variation in fold assignment.

1.3.4 Missing-value handling

The dataset was checked for missing values before model fitting.

```
cat("Total missing values:", sum(is.na(analysis_data)), "\n")
```

```
## Total missing values: 0
```

No missing values were detected; therefore, no imputation was required. If missing values had been present, imputation statistics would have been estimated using only the training data and then applied unchanged to the test data.

1.3.5 Leakage control

Several safeguards were implemented to prevent information leakage throughout the analysis:

- The train–test split was performed before any preprocessing.
- Predictor standardization used statistics estimated from the training data only.
- Five-fold cross-validation was conducted exclusively within the training set.
- Hyperparameter tuning for ridge, lasso, and elastic net relied solely on training cross-validation error.
- The holdout responses (`y_test`) were not accessed until the final evaluation stage.

As a result, the holdout test set provides an unbiased assessment of predictive performance because no information from the test observations entered the model construction or tuning process.

1.4 1C. Training-data exploration

1.5 Pairwise correlation among predictors

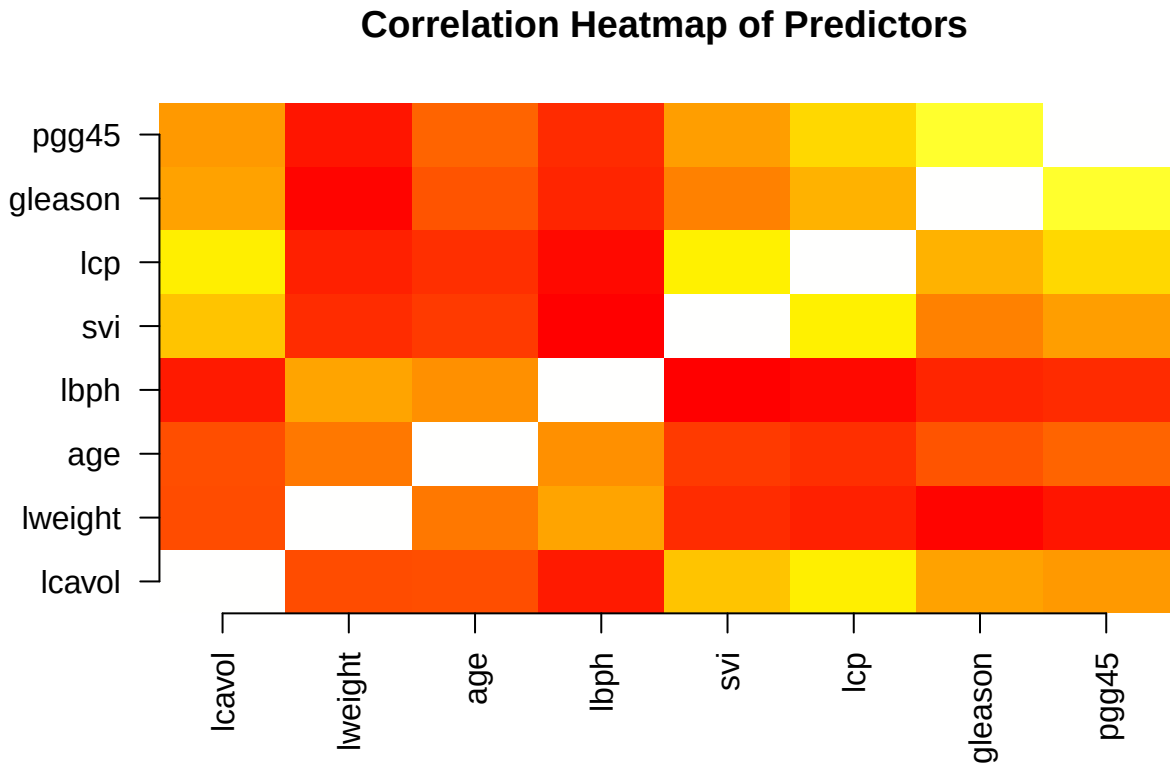
```
# Pairwise correlation among predictors
cor_matrix <- cor(
  train_df[, predictors],
  use = "complete.obs"
)

knitr::kable(
  round(cor_matrix, 2),
  caption = "Pairwise correlation matrix among predictors"
)
```

Table 2: Pairwise correlation matrix among predictors

	lcavol	lweight	age	lbph	svi	lcp	gleason	pgg45
lcavol	1.00	0.17	0.17	0.01	0.55	0.68	0.44	0.41
lweight	0.17	1.00	0.31	0.44	0.07	0.03	-0.06	-0.01
age	0.17	0.31	1.00	0.38	0.11	0.08	0.19	0.24
lbph	0.01	0.44	0.38	1.00	-0.07	-0.04	0.05	0.06
svi	0.55	0.07	0.11	-0.07	1.00	0.69	0.33	0.43
lcp	0.68	0.03	0.08	-0.04	0.69	1.00	0.49	0.61
gleason	0.44	-0.06	0.19	0.05	0.33	0.49	1.00	0.78
pgg45	0.41	-0.01	0.24	0.06	0.43	0.61	0.78	1.00

```
# Correlation heatmap
cor_matrix <- cor(train_df[, predictors], use = "complete.obs")
```

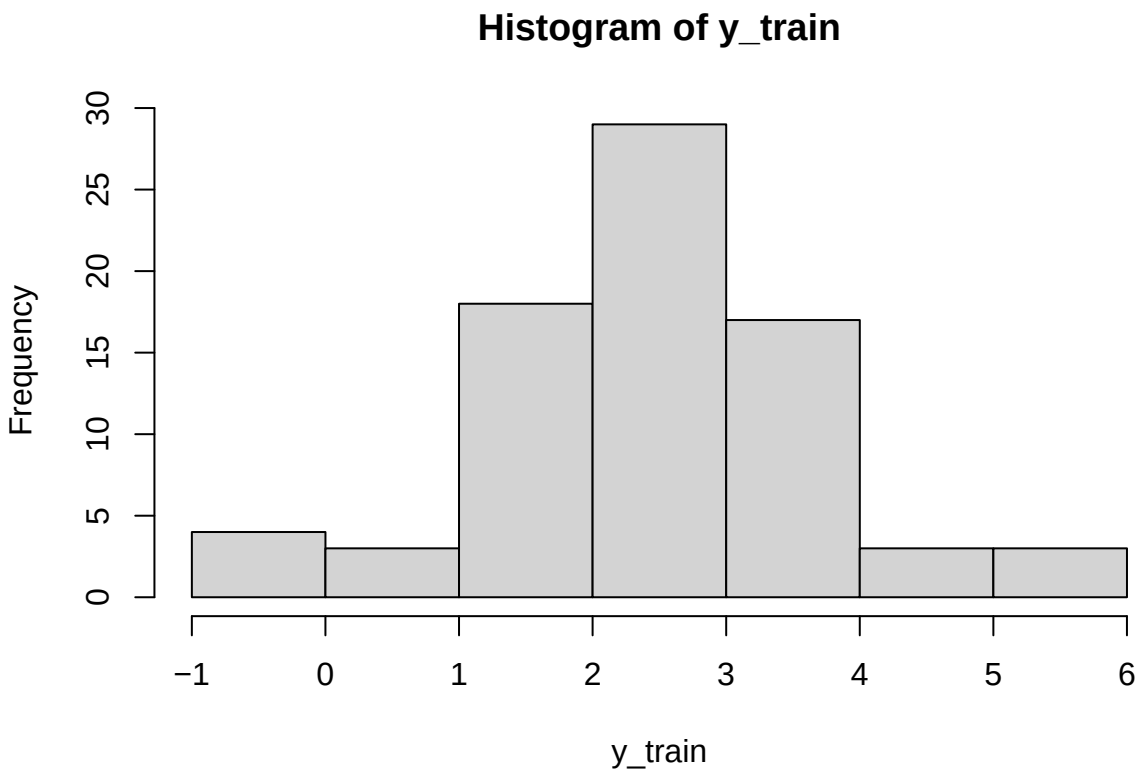
```r
image(1:ncol(cor_matrix), 1:ncol(cor_matrix), cor_matrix,
      axes = FALSE, xlab = "", ylab = "",
      col = heat.colors(1000), main = "Correlation Heatmap of Predictors")
axis(1, at = 1:ncol(cor_matrix), labels = colnames(cor_matrix), las = 2)
axis(2, at = 1:ncol(cor_matrix), labels = colnames(cor_matrix), las = 1)
```



The pairwise correlation matrix was examined to identify potential relationships among predictors. Most predictors showed weak to moderate correlations, suggesting limited redundancy between features. However, some stronger positive correlations were observed, particularly between gleason and pgg45 ($r = 0.78$), lcavol and lcp ($r = 0.68$), and svi and lcp ($r = 0.69$). These relationships indicate that some predictors may share related clinical information. Therefore, Variance Inflation Factor (VIF) analysis was further performed to assess the overall multicollinearity effect in the regression model.

1.6 Response Distribution

```r
hist(y_train)
```




```
summary(y_train)
```

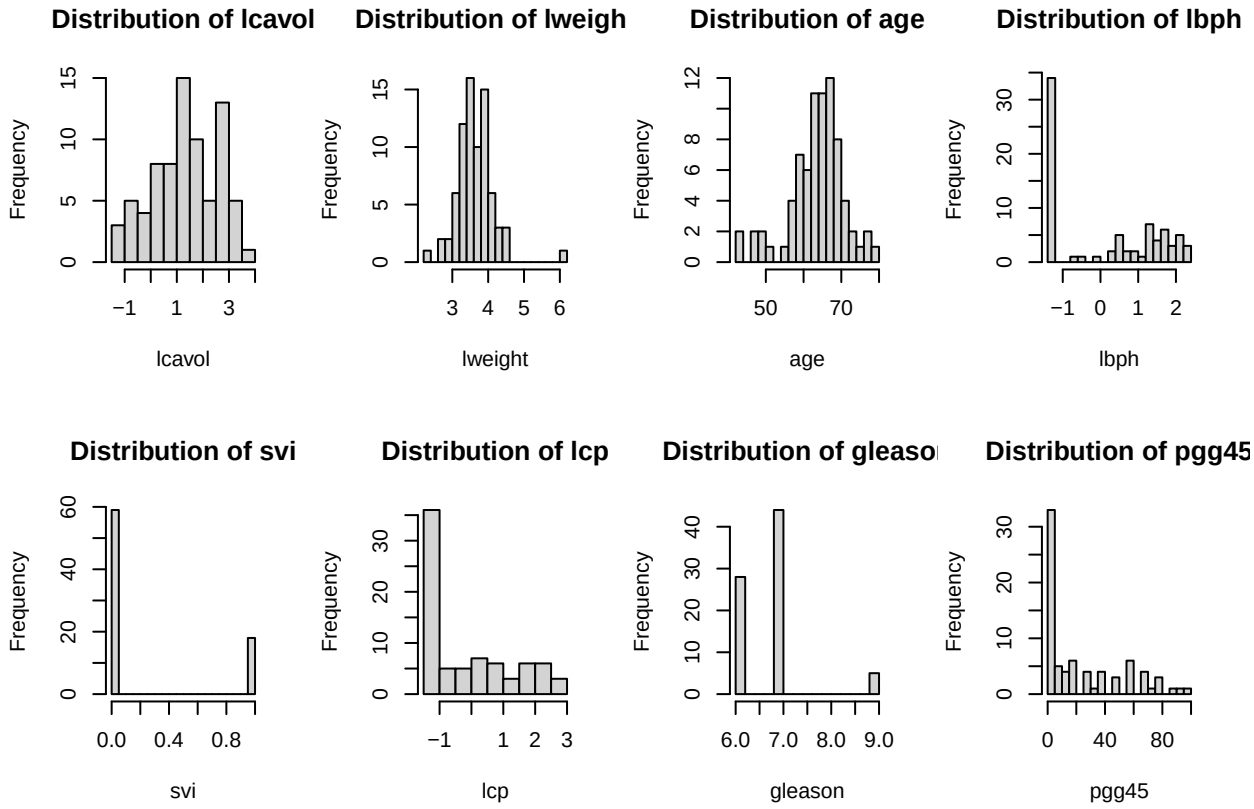
```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## -0.4308  1.7664  2.5688  2.4757  3.2753  5.5829
```

The distribution of the response variable (lpsa) was examined using a histogram and summary statistics. The response values range from -0.43 to 5.58, with a median of 2.57 and a mean of 2.48. The mean and median are relatively close, indicating that the distribution is approximately symmetric with no strong skewness. Although some observations appear at the lower and higher ends of the range, no severe outliers are evident from the overall distribution. Therefore, no transformation was applied to the response variable before model training.

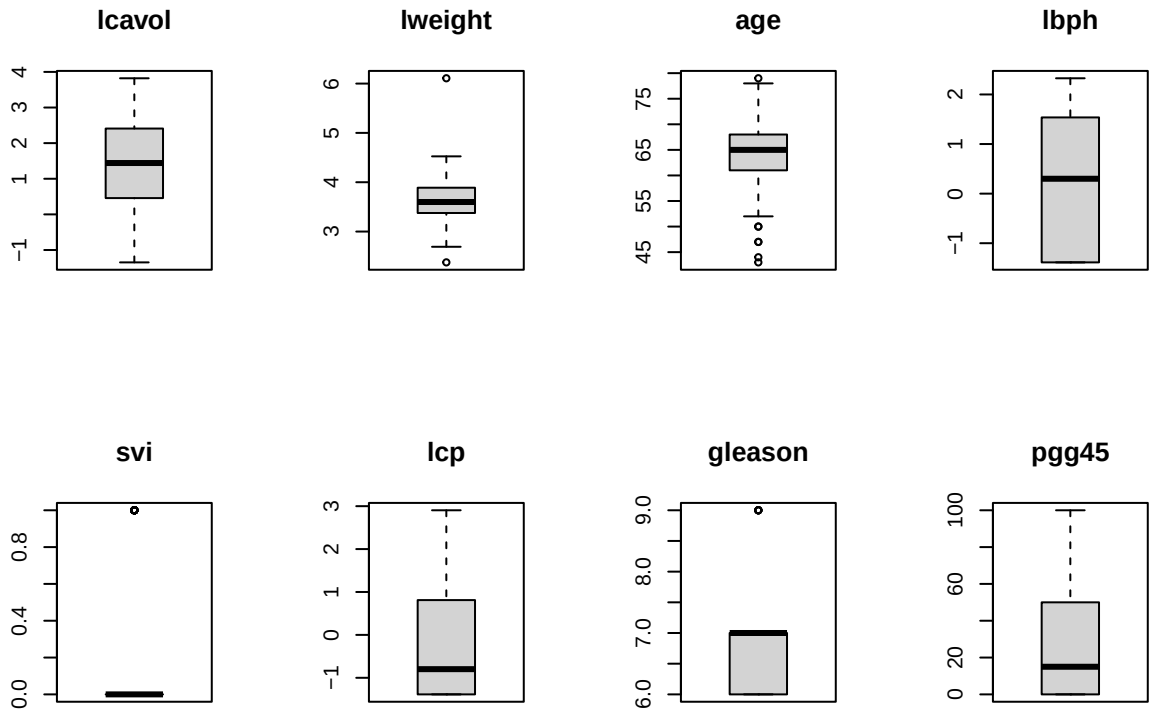
1.6.1 Predictor Histogram & Boxplot

```
## Histogram
par(mfrow = c(2,4))

for (var in predictors) {
  hist(
    train_df[[var]],
```

```
    main = paste("Distribution of", var),
    xlab = var,
    breaks = 15
  )
}
```




```
par(mfrow = c(1,1))

## Boxplot
par(mfrow = c(2,4))
for (var in predictors) {
  boxplot(
    train_df[[var]],
    main = var
  )
}
```




```
par(mfrow = c(1,1))
```

The histograms show that most predictors have different distribution patterns. lcavol, lweight, and age are approximately symmetric, while lbph, lcp, and pgg45 exhibit right-skewed distributions with longer tails toward higher values. svi and gleason are discrete variables with highly concentrated values. These distribution characteristics are considered in later regression analysis and model diagnostics.

Upon inspecting the visualizations, a potential influential observation is evident in the lweight vs lpsa plot. There is a single data point located at the far right with an exceptionally high lweight (exceeding 6.0) but a relatively low lpsa (approximately 2.0).

1.6.2 Correlation between predictors and response

```
cor_response <- cor(
  train_df[, predictors],
  train_df$lpsa
)

knitr::kable(
  round(cor_response, 2),
```

```
  caption = "Correlation between predictors and lpsa"
)
```

Table 3: Correlation between predictors and lpsa

lcavol	0.77
lweight	0.30
age	0.14
lbph	0.14
svi	0.57
lcp	0.59
gleason	0.39
pgg45	0.41

The correlation analysis indicates that all predictors have positive correlations with lpsa. lcavol shows the strongest relationship (r = 0.77), followed by lcp (r = 0.59) and svi (r = 0.57). Variables such as pgg45, gleason, and lweight have moderate correlations, while age and lbph show weak associations. Therefore, lcavol, lcp, and svi are likely to be important predictors in the multiple linear regression model.

1.6.3 Check multicollinearity using VIF

```
vif_model <- lm(
  lpsa ~ .,
  data = train_df[, c("lpsa", predictors)]
)

vif_values <- vif(vif_model)

knitr::kable(
  round(vif_values, 2),
  caption = "Variance Inflation Factor (VIF)"
)
```

Table 4: Variance Inflation Factor (VIF)

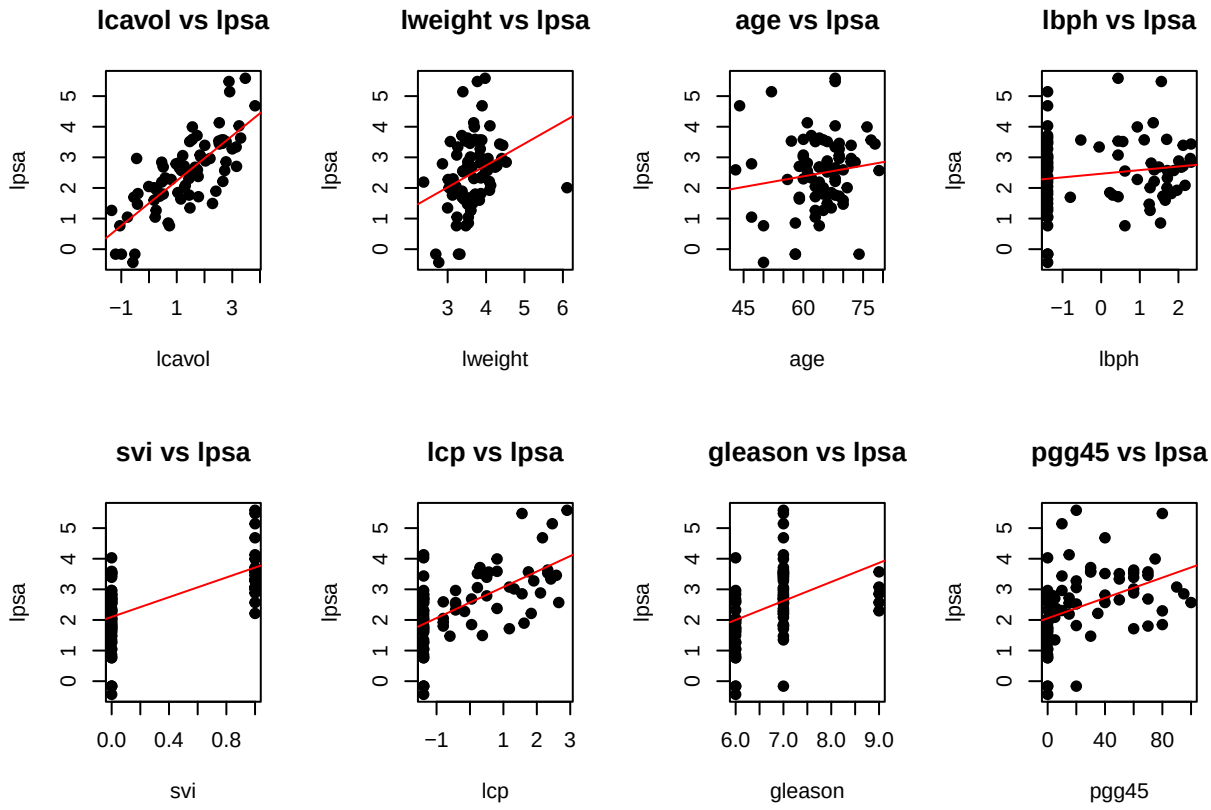
	x
lcavol	2.18
lweight	1.38
age	1.32
lbph	1.40
svi	1.97

	x
lcp	3.29
gleason	2.79
pgg45	3.35

There is no significant multicollinearity present in this model. Because all VIF values are safely below 5, the independent variables are not highly correlated with one another. The regression model's coefficient estimates are mathematically stable, and no variables need to be dropped or transformed for multicollinearity reasons.

1.6.4 Predictor-response plots

```
par(mfrow = c(2,4))

for (var in predictors) {
  plot(
    train_df[[var]],
    train_df$lpsa,
    xlab = var,
    ylab = "lpsa",
    main = paste(var, "vs lpsa"),
    pch = 19
  )

  abline(
    lm(train_df$lpsa ~ train_df[[var]]),
    col = "red"
  )
}
```




```
par(mfrow = c(1,1))
```

The provided scatter plots visualize the marginal relationships between the eight candidate predictors and the target variable, lpsa.

Strong Trends: lcavol demonstrates the clearest and most distinct positive linear correlation with lpsa, suggesting it will be a strong predictor in the model.

Categorical Patterns: Variables such as svi (binary) and gleason (ordinal) display expected clustered structures, with higher categories generally corresponding to higher lpsa values.

Weak Trends: age and lbph exhibit highly dispersed data clouds with very flat regression lines, indicating weak independent linear relationships with the target.

1.6.4.1 Question Given the varying individual predictive strengths of these features and the moderate structural correlations (e.g., between pgg45 and lcp), will an L_1 penalty (Lasso) completely eliminate the structurally weaker predictors like age to produce a strictly sparse model, or will an L_2 penalty (Ridge) prove more stable against the identified high-leverage anomaly in lweight?

2 Problem 2. OLS, ridge, and lasso

2.1 2A. OLS baseline

```
# Convert x_train (matrix) and y_train into a data frame for lm()
train_data <- data.frame(y = y_train, x_train)

# 1. Fit OLS on the training set
ols_fit <- lm(y ~ ., data = train_data)

# 2. Extract coefficients (tidy table via broom)
ols_coefs <- broom::tidy(ols_fit)
kable(
  ols_coefs,
  digits = 4,
  caption = "OLS coefficient estimates on standardized training data."
)
```

Table 5: OLS coefficient estimates on standardized training data.

term	estimate	std.error	statistic	p.value
(Intercept)	2.4757	0.0826	29.9860	0.0000
lcavol	0.7228	0.1220	5.9243	0.0000
lweight	0.2022	0.0971	2.0834	0.0410
age	-0.1305	0.0949	-1.3751	0.1736
lbph	0.1336	0.0978	1.3657	0.1765
svi	0.2749	0.1160	2.3699	0.0206
lcp	-0.0413	0.1497	-0.2757	0.7836
gleason	0.0365	0.1380	0.2644	0.7923
pgg45	0.0945	0.1510	0.6257	0.5336

```
# 3. R-squared and Adjusted R-squared from the model summary
ols_summary <- summary(ols_fit)
cat(sprintf("R-squared:          %.4f\n", ols_summary$r.squared))
```

```
## R-squared:          0.6729
```

```
cat(sprintf("Adjusted R-squared: %.4f\n", ols_summary$adj.r.squared))
```

```
## Adjusted R-squared: 0.6345
```

```r
# 4. Predict on the training set and evaluate training error
ols_train_pred <- predict(ols_fit, newdata = train_data)
ols_train_metrics <- score_regression(y = y_train, prediction = ols_train_pred)
cat(sprintf("Training RMSE:      %.4f\n", ols_train_metrics["RMSE"]))
```

```
## Training RMSE:      0.6808
```

```r
cat(sprintf("Training MAE:       %.4f\n", ols_train_metrics["MAE"]))
```

```
## Training MAE:       0.5170
```

```r
# 5. Five-fold CV error using the shared foldid with ridge/lasso
#    (lm() has no built-in CV, so we compute it manually)
K <- max(foldid)
cv_mse_folds <- numeric(K)
for (k in seq_len(K)) {
  idx_train_k <- which(foldid != k)
  idx_val_k   <- which(foldid == k)
  fold_data   <- data.frame(y = y_train[idx_train_k], x_train[idx_train_k, , drop
 ↪  = FALSE])
  fold_fit    <- lm(y ~ ., data = fold_data)
  fold_pred   <- predict(fold_fit,
                         newdata = data.frame(x_train[idx_val_k, , drop =
                          ↪  FALSE]))
  cv_mse_folds[k] <- mean((y_train[idx_val_k] - fold_pred)^2)
}
ols_cv_mse  <- mean(cv_mse_folds)
ols_cv_rmse <- sqrt(ols_cv_mse)
cat(sprintf("5-fold CV MSE:      %.4f\n", ols_cv_mse))
```

```
## 5-fold CV MSE:      0.6297
```

```r
cat(sprintf("5-fold CV RMSE:     %.4f\n", ols_cv_rmse))
```

```
## 5-fold CV RMSE:     0.7935
```

```r
# 6. Residual / influence diagnostic: Cook's distance
cooks_d   <- cooks.distance(ols_fit)
n_train   <- nrow(train_data)
threshold <- 4 / n_train

plot(cooks_d,
     type = "h", lwd = 1.5, col = "steelblue",
```

```
     ylab = "Cook's distance", xlab = "Observation index",
     main = "Cook's Distance - OLS Baseline")
abline(h = threshold, lty = 2, col = "firebrick", lwd = 1.5)
text_idx <- which(cooks_d > threshold)
if (length(text_idx)) {
  text(text_idx, cooks_d[text_idx],
       labels = text_idx, pos = 3, cex = 0.8, col = "firebrick")
}
```

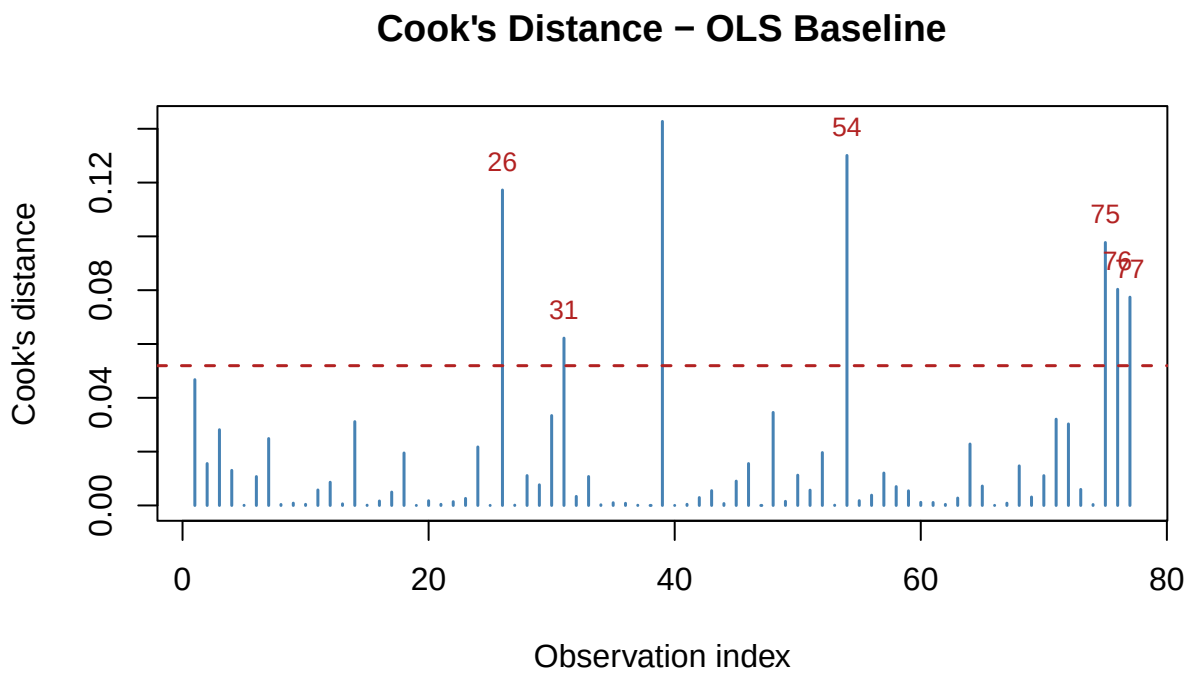


Figure 1: Cook's distance for the OLS fit. Observations exceeding the 4/n threshold (dashed line) are flagged as influential.

```
cat(sprintf("Threshold (4/n):    %.4f\n", threshold))
```

```
## Threshold (4/n):    0.0519
```

```
cat(sprintf("Influential points: %d of %d\n", length(text_idx), n_train))
```

```
## Influential points: 7 of 77
```

Model summary. The OLS model is fitted on the 77 standardized training observations with all 8 predictors. Because every predictor has been centred and scaled to unit (population) variance,

the intercept equals the mean of `lpsa` in the training set, and each slope measures how much `lpsa` changes per one population-SD increase in that predictor, holding the remaining seven fixed.

Coefficient interpretation. Two predictors dominate.

- `lcavol` carries the largest standardized coefficient ($\hat{\beta} \approx 0.72$, $p < 0.001$), confirming that log cancer volume is the single strongest linear predictor of log PSA.
- `svi` is the second-largest contributor ($\hat{\beta} \approx 0.27$, $p \approx 0.021$), indicating that seminal vesicle invasion is associated with meaningfully higher PSA even after adjusting for tumour volume.
- `lweight` is marginally significant ($p \approx 0.041$); the remaining 5 predictors (`age`, `lbph`, `lcp`, `gleason`, `pgg45`) all have $|t| < 1.4$ and $p > 0.16$.

Collinearity concern. The near-zero and sign-reversed coefficient for `lcp` ($\hat{\beta} \approx -0.04$) is noteworthy: biologically, greater capsular penetration should correlate positively with PSA, and indeed the marginal correlation is positive. The sign flip in OLS is a classic symptom of multicollinearity — `lcavol` and `lcp` are strongly correlated ($r \approx 0.68$ on the training set), so OLS distributes their shared variance unevenly. Regularization in 2B–2C is expected to stabilize these shared-variance coefficients.

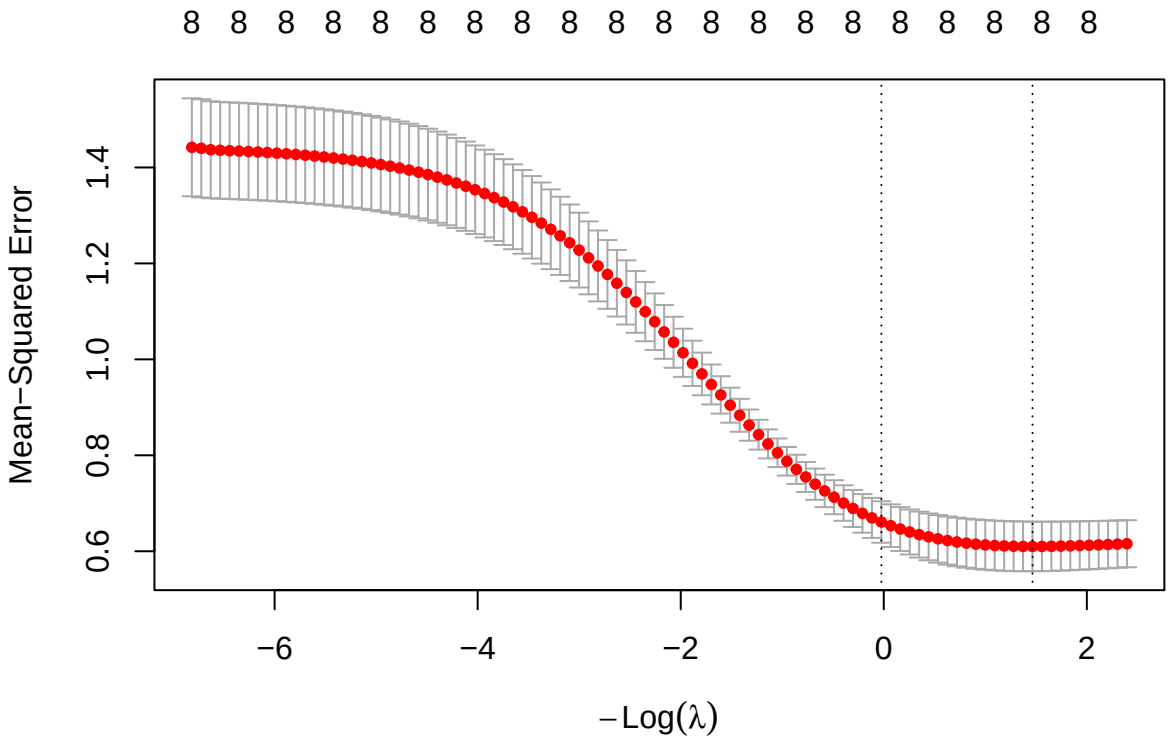
Cross-validation error. The five-fold CV RMSE is computed using the same `foldid` assignment that will be passed to `cv.glmnet` for ridge and lasso, ensuring a like-for-like comparison. Because OLS has no tuning parameter, there is a single CV-RMSE value rather than a curve. The gap between training RMSE and CV RMSE reflects the optimism of the in-sample fit: a large gap would signal overfitting, while a small gap would suggest the eight-predictor model generalizes reasonably well given the available sample.

Influence diagnostic. The Cook's distance plot flags observations whose removal would substantially shift the fitted coefficients. The conventional threshold $4/n$ is shown as a dashed line. Any point above this line has outsized leverage, large residual, or both. In a sample of only 77 training rows, even one or two influential points can destabilize an unpenalized fit — particularly for the collinear predictors `lcavol`/`lcp` whose coefficient estimates are already fragile. These influential observations are retained in the analysis (not removed) because they are clinically valid patients; their effect is instead mitigated by the regularization introduced in 2B–2C.

2.2 2B. Ridge

2.2.1 Cross-validation curve:

```
ridge <- fit_cv_glmnet(x = x_train, y = y_train, alpha=0, foldid = foldid)
plot(ridge)
```



The mean cross-validation MSE decreases as $-Log(\lambda)$ increases (equivalently, as λ decreases), suggesting that reducing the regularization strength improves predictive performance. The left dashed line indicates λ_{1se}, which selects a more regularized model with prediction error within one standard error of the minimum, while the right dashed line indicates λ_{min}, which yields the lowest cross-validation error.

2.2.2 Lambda min:

```
g <- crossprod(x_train)
eigen <- eigen(g)$value

lambda <- ridge$lambda.min
cat("Lambda min: ",lambda, "\n")
```

```
## Lambda min:  0.2311277
```

```
cat("MSE: ", ridge$cvm[ridge$lambda == lambda], "\n")
```

```
## MSE:  0.6100275
```

```r
cat("Condition number:", (max(eigen) + lambda)/(min(eigen) + lambda), "\n")
```

```
## Condition number: 20.22951
```

With $\lambda_{min} \approx 0.23$, the 5-fold cross-validation MSE reaches its minimum value of $MSE_{min} \approx 0.61$, indicating the best predictive performance among the candidate values of λ. However, the corresponding Ridge model still has a relatively large condition number, suggesting that the model remains sensitive to multicollinearity and may not be sufficiently stable.

2.2.3 Lambda 1se:

```r
g <- crossprod(x_train)
eigen <- eigen(g)$value

lambda <- ridge$lambda.1se
cat("Lambda 1se: ",lambda, "\n")
```

```
## Lambda 1se:  1.024039
```

```r
cat("MSE: ", ridge$cvm[ridge$lambda == lambda], "\n")
```

```
## MSE:  0.6609706
```

```r
cat("Condition number:", (max(eigen) + lambda)/(min(eigen) + lambda), "\n")
```

```
## Condition number: 19.06877
```

Although the one-standard-error rule selected a larger penalty $\lambda_{1se} \approx 1.02$, the condition number decreased only slightly from 20.2 to 19.07, indicating minimal improvement in numerical stability. In contrast, the cross-validation MSE increased from 0.61 to 0.66. Therefore, the additional regularization provided by λ_{1se} offers little practical benefit while reducing predictive performance.

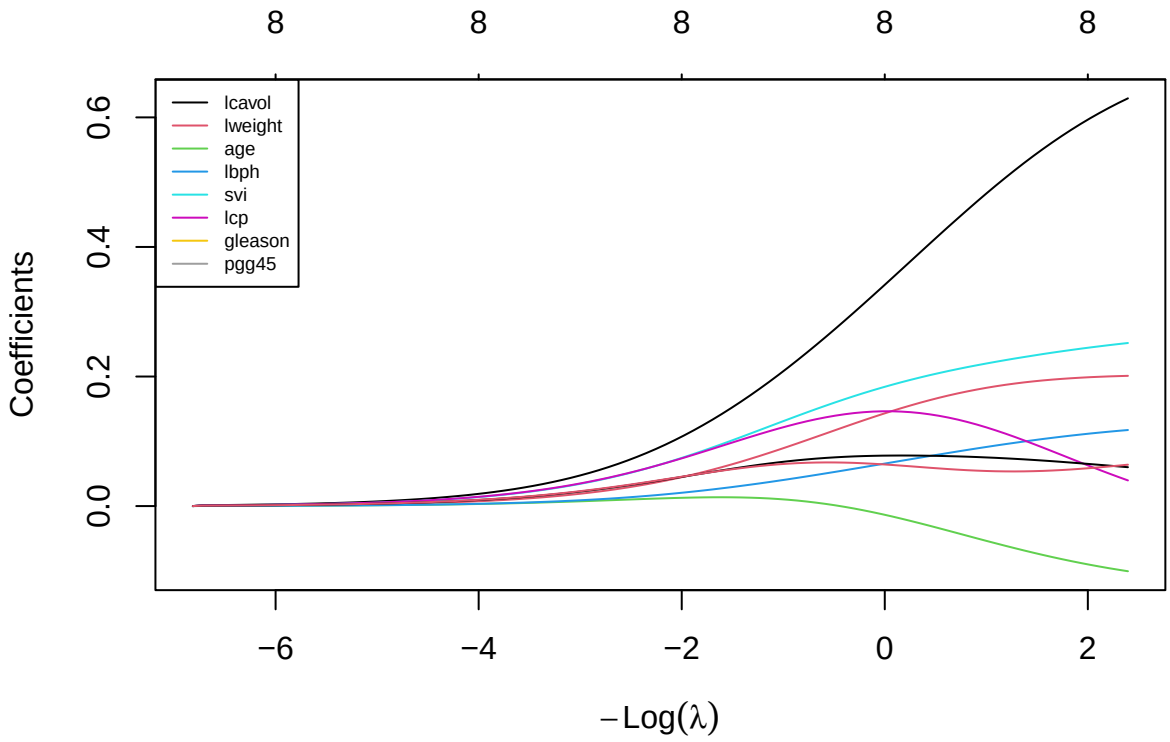
2.2.4 Coefficients:

```r
data.frame(
  OLS = as.numeric(coef(ols_fit)),
  Lambda_Min = as.numeric(coef(ridge, "lambda.min")),
  Lambda_Max = as.numeric(coef(ridge, "lambda.1se")),
  row.names = names(ols_fit$coefficients)
)
```

```
##                     OLS  Lambda_Min  Lambda_Max
## (Intercept)  2.47569468  2.47569468  2.47569468
## lcavol       0.72282252  0.54055005  0.33822482
## lweight      0.20222141  0.19253291  0.14197627
## age         -0.13046518 -0.07201800 -0.01262288
## lbph         0.13361911  0.10166159  0.06498686
## svi          0.27494937  0.23239051  0.18307566
## lcp         -0.04125506  0.09593321  0.14631601
## gleason      0.03648450  0.07123914  0.07784505
## pgg45        0.09450125  0.05388195  0.06447679
```

Compared with the OLS model, Ridge regression shrinks most regression coefficients toward zero, reducing their magnitudes through the regularization penalty. This shrinkage helps alleviate the effects of multicollinearity while retaining all predictors in the model. Most coefficients exhibit the expected behavior by moving closer to zero as the regularization strength increases. However, two coefficients show notable exceptions. The coefficient of **lcp** changes from a small negative value in the OLS model to positive values under Ridge regression. In addition, the coefficient of **gleason** increases in magnitude, moving further away from zero rather than shrinking like the other coefficients. These changes suggest that the effects of **lcp** and **gleason** are influenced by multicollinearity with other predictors. As Ridge simultaneously estimates all coefficients under a penalty, it redistributes the effects among correlated variables, causing some coefficients to increase in magnitude or even change sign despite the overall shrinkage trend.

```r
n_vars <- nrow(ridge$glmnet.fit$beta)
plot(ridge$glmnet.fit)
legend("topleft",
       legend = rownames(ridge$glmnet.fit$beta),
       col = 1:n_vars,
       lty = 1,
       cex = 0.6)
```



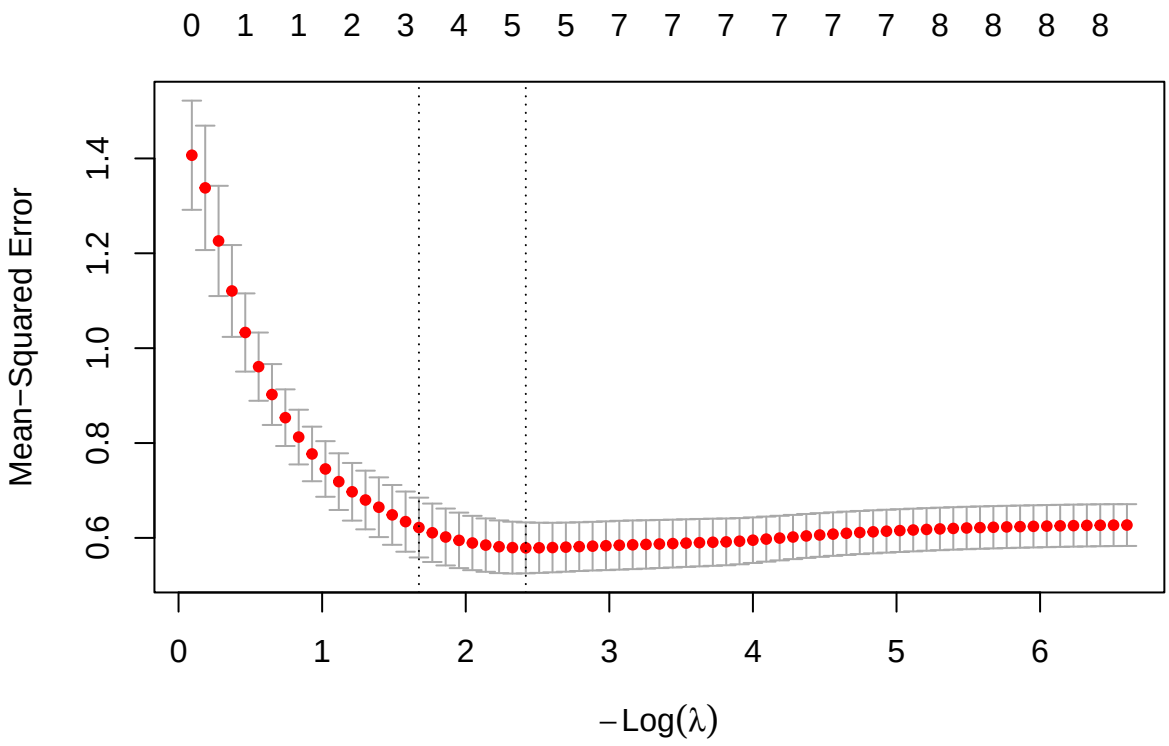
As λ increases, all regression coefficients are progressively shrunk toward zero, demonstrating the regularization effect of Ridge regression. Conversely, as λ decreases, the coefficients gradually move away from zero toward their OLS estimates. The coefficient of **lcavol** remains the largest throughout the regularization path, indicating that it has the strongest association with the response variable. The coefficients of **svi** and lweight also increase substantially as the regularization weakens, suggesting relatively strong effects on the response. In contrast, the coefficients of **gleason**, **lbph**, and **pgg45** remain relatively small across the entire path, indicating weaker contributions to the model. The coefficient of age remains negative throughout the regularization path, reflecting a consistent negative relationship with the response variable. Notably, the coefficient of **lcp** exhibits non-monotonic behavior, first increasing and then decreasing as λ changes, suggesting that its estimated effect is sensitive to multicollinearity with other predictors.

To address multicollinearity, Ridge impose an L_2 penalty on the regression coefficients. Rather than assigning a large coefficient to one correlated predictor and a small coefficient to another, Ridge shrinks the coefficients simultaneously and redistributes their effects among the correlated variables. Consequently, the coefficient estimates become more stable and less sensitive to small changes in the data.

2.3 2C. Lasso

2.3.1 Model fitting and cross-validation

```
lasso_cv <- fit_cv_glmnet(
    x = x_train,
    y = y_train,
    alpha = 1,
    foldid = foldid
)
plot(lasso_cv)
```



Based on the 5-fold cross-validation curve generated for the Lasso regression model ($\alpha = 1$), we observe the relationship between the tuning parameter λ and the Mean-Squared Error (MSE). The top axis indicates the number of active (non-zero) predictors retained in the model at each level of penalization.

- The Cross-Validation Curve: The red dots represent the average CV-MSE across the 5 folds for each λ value, with the gray error bars denoting one standard error above and below the mean. As $-\log(\lambda)$ increases (moving from left to right, meaning the penalty weakens), the MSE initially drops, reaches a minimum, and then slightly stabilizes as the model approaches the unpenalized OLS limit.

- Optimal λ (λ_{min}): The right vertical dashed line marks `lambda.min`, the value that strictly minimizes the cross-validation MSE. At this optimal point for prediction accuracy ($-\log(\lambda) \approx 2.4$), the Lasso penalty retains 5 non-zero predictors*.
- Parsimonious λ (λ_{1se}): The left vertical dashed line marks `lambda.1se`, selected by the one-standard-error rule. This value applies a stronger penalty ($-\log(\lambda) \approx 1.67$) to produce a sparser model. It retains only 3 non-zero predictors while keeping the error within one standard deviation of the absolute minimum MSE, offering a robust trade-off between predictive power and model simplicity.

2.3.2 Analysis of λ_{min}

```
lambda_min <- lasso_cv$lambda.min
coef(lasso_cv, s="lambda.min")
```

```
## 9 x 1 sparse Matrix of class "dgCMatrix"
##              lambda.min
## (Intercept) 2.47569468
## lcavol      0.67611312
## lweight     0.12774355
## age         .
## lbph        0.03085712
## svi         0.20454921
## lcp         .
## gleason     .
## pgg45       0.03234366
```

```
count_nonzero(lasso_cv, s="lambda.min")
```

```
## [1] 5
```

The optimal tuning parameter selected by five-fold cross-validation is $\lambda_{\text{min}} = 0.0891$. This value minimizes the cross-validation Mean Squared Error (CV-MSE), providing the best predictive performance among all candidate penalty values.

At this level of regularization, the Lasso model retains five non-zero predictors: lcavol, lweight, lbph, svi, and pgg45. The remaining predictors (age, lcp, and gleason) are shrunk exactly to zero, demonstrating Lasso's ability to perform automatic variable selection while maintaining predictive accuracy.

The retained predictors suggest that tumor volume (lcavol) is the most influential variable, having the largest estimated coefficient (0.6761). Variables related to body weight (lweight), benign prostatic hyperplasia (lbph), seminal vesicle invasion (svi), and the percentage of Gleason grades 4 or 5 (pgg45) also contribute to predicting PSA levels, although with relatively smaller effects.

Overall, λ_{min} produces the model with the lowest cross-validation error while preserving a moderate number of predictors, offering a balance between prediction accuracy and model complexity.

```r
cv_mse_min <- lasso_cv$cvm[
    lasso_cv$lambda==
    lasso_cv$lambda.min
]
cv_mse_min
```

```
## [1] 0.5789934
```

The minimum five-fold cross-validation Mean Squared Error (CV-MSE) achieved by the Lasso model is **0.579**. This value corresponds to $\lambda_{\text{min}} = 0.0891$ and represents the estimated prediction error obtained during model tuning using the training data.

A lower CV-MSE indicates better expected predictive performance on unseen data. Since λ_{min} is selected to minimize this metric, it is the tuning parameter that prioritizes prediction accuracy over model simplicity.

2.3.3 Analysis of λ_{1se}

```r
lambda_1se <- lasso_cv$lambda.1se
coef(lasso_cv, s="lambda.1se")
```

```
## 9 x 1 sparse Matrix of class "dgCMatrix"
##             lambda.1se
## (Intercept) 2.47569468
## lcavol      0.63661195
## lweight     0.05343261
## age         .
## lbph        .
## svi         0.14411468
## lcp         .
## gleason     .
## pgg45       .
```

```r
count_nonzero(lasso_cv, s="lambda.1se")
```

```
## [1] 3
```

The one-standard-error rule selects $\lambda_{\text{1se}} = 0.1875$, which applies a stronger regularization penalty than λ_{min}. The corresponding five-fold cross-validation Mean Squared Error (CV-MSE) is r round(cv_mse_1se, 4), remaining within one standard error of the minimum while producing a simpler model.

Under λ_{1se}, the Lasso model retains only three non-zero predictors (lcavol, lweight, and svi). Compared with λ_{min}, the variables lbph and pgg45 are removed, resulting in a more parsimonious and interpretable model with only a small loss in predictive performance.

```
cv_mse_1se <- lasso_cv$cvm[
    lasso_cv$lambda==
    lasso_cv$lambda.1se
]
cv_mse_1se
```

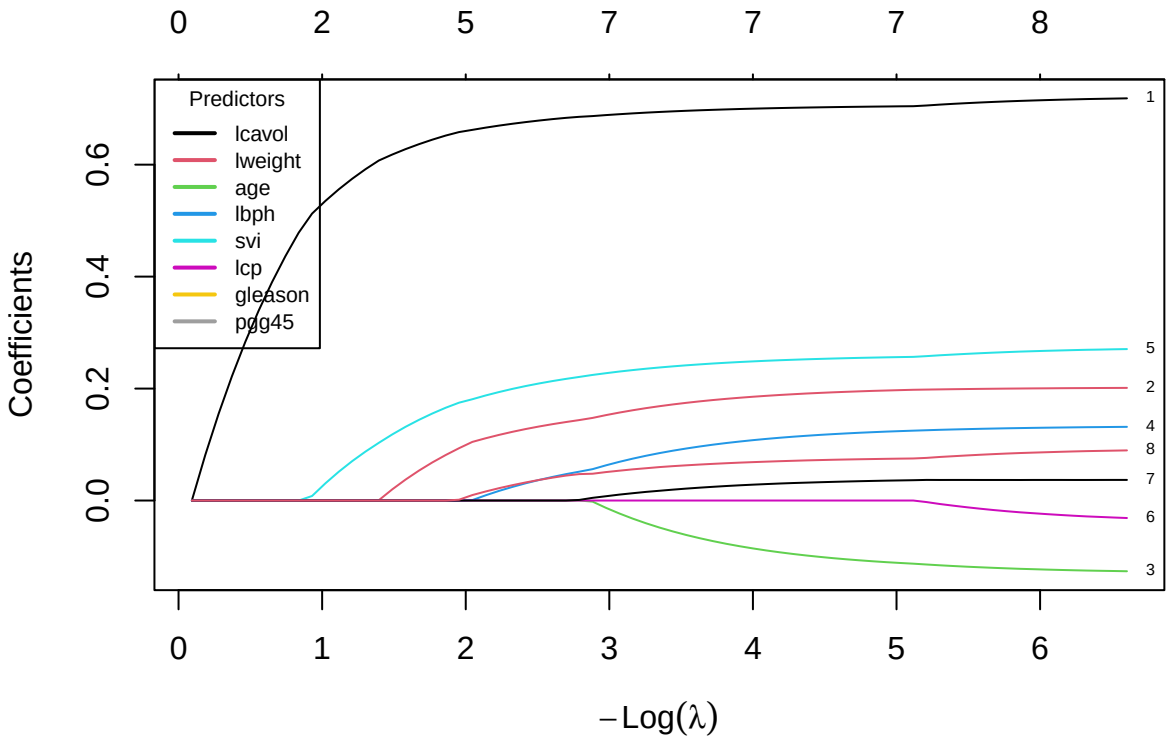
```
## [1] 0.6217887
```

The one-standard-error rule selects $\lambda_{1se} = rround(lambda_1 se, 4)$, resulting in a five-fold cross-validation Mean Squared Error (CV-MSE) of r round(cv_mse_1se, 4). Although this error is slightly higher than that of λ_{min}, it remains within one standard error of the minimum while providing a more conservative model.

Under λ_{1se}, the Lasso model retains only three non-zero predictors (lcavol, lweight, and svi). Compared with λ_{min}, the predictors lbph and pgg45 are removed, resulting in a simpler and more interpretable model with only a modest reduction in predictive performance.

2.3.4 Coefficient Path

```
lasso_fit <- glmnet(
    x_train,
    y_train,
    alpha=1,
    standardize=FALSE,

)

plot(lasso_fit, xvar = "lambda", label = TRUE)

legend("topleft",
       legend = rownames(lasso_fit$beta),
       col = 1:nrow(lasso_fit$beta),
       lty = 1,
       cex = 0.7,
       lwd = 2,
       title = "Predictors")
```



Overall, the coefficient path shows that as $-\log(\lambda)$ increases (equivalently, as λ decreases), the regularization penalty becomes weaker, allowing more predictors to enter the model and their coefficients to increase in magnitude. Under strong regularization, many coefficients are shrunk exactly to zero, while weaker regularization gradually recovers the full model.

Among the predictors, **lcavol** enters the model first and consistently has the largest coefficient, indicating that it is the most influential predictor of **lpsa**. **svi** and **lweight** also become active relatively early and maintain positive coefficients throughout the path. In contrast, **age, lbph, lcp, gleason,** and **pgg45** enter only when the penalty becomes sufficiently small, suggesting that their contributions are comparatively weaker and less stable.

2.4 2D

```
# Build a comparison from training/CV quantities only.
```

Core model nominated before holdout evaluation: [method and reason]

2.5 2E. Perturbation or stability check

Prediction before running the check: [prediction]

```
# Implement one allowed training-only stability check.
```

Observed result and explanation: [result]

3 Problem 3. Mathematical mechanisms

Let

$$X \in \mathbb{R}^{n \times p}$$

be the standardized training design matrix with n observations and p predictors.

Let

$$y \in \mathbb{R}^{n}$$

be the response vector and

$$b \in \mathbb{R}^{p}$$

be the coefficient vector.

3.1 3A. Ridge and conditioning

Write the ridge derivation. Define dimensions and explain uniqueness.

```
# Use x_train and the fitted ridge lambda.min.
# safe_condition_numbers(x_train, lambda = ...)
```

3.2 3A. Ridge and conditioning

3.2.1 Ridge objective

The ridge objective function is

$$Q_{\lambda}(b) = \frac{1}{2n}\|y - Xb\|^2 + \frac{\lambda}{2}\|b\|^2.$$

Taking the gradient,

$$\nabla Q_{\lambda}(b) = \frac{1}{n}(X^T Xb - X^T y) + \lambda b.$$

Setting the gradient equal to zero,

$$\frac{1}{n}X^T Xb - \frac{1}{n}X^T y + \lambda b = 0.$$

Rearranging,

$$\left(\frac{1}{n}X^T X + \lambda I\right) b = \frac{1}{n}X^T y.$$

Define

$$G = \frac{1}{n}X^T X, \qquad z = \frac{1}{n}X^T y.$$

Hence,

$$(G + \lambda I)b = z,$$

and therefore

$$\boxed{\hat{b} = (G + \lambda I)^{-1} z.}$$

Since G is positive semidefinite and $\lambda > 0$,

$$G + \lambda I$$

is positive definite and therefore invertible. Hence the ridge estimator is unique.
The spectral condition number is

$$\kappa_2(G) = \frac{\mu_{\max}}{\mu_{\min}},$$

while ridge gives

$$\kappa_2(G + \lambda I) = \frac{\mu_{\max} + \lambda}{\mu_{\min} + \lambda}.$$

where

$$\mu_{\max} = \max_i \mu_i$$

is the **largest eigenvalue** of the Gram matrix G, and

$$\mu_{\min} = \min_i \mu_i$$

is the **smallest (positive) eigenvalue** of G.

3.2.2 Condition Number and Numerical Stability

The Gram matrix is defined as

$$G = \frac{1}{n} X^T X.$$

Since G is symmetric and positive semidefinite, all of its eigenvalues are non-negative. Let the eigenvalues of G be

$$\mu_1, \mu_2, \dots, \mu_p,$$

where

$$\mu_{\max} = \max_i \mu_i, \qquad \mu_{\min} = \min_i \mu_i.$$

The spectral condition number is defined as

$$\kappa_2(G) = \frac{\mu_{\max}}{\mu_{\min}}.$$

This quantity measures how sensitive the solution of a linear system is to small perturbations in the input data. A large condition number indicates that the Gram matrix is close to singular, so the estimated coefficients may change dramatically even when the training data are only slightly perturbed. Conversely, a small condition number indicates that the matrix is well-conditioned and the regression coefficients are numerically stable.

Ridge regression replaces the Gram matrix by

$$G + \lambda I.$$

If the eigenvalues of G are

$$\mu_1, \mu_2, \dots, \mu_p,$$

then the eigenvalues of the ridge matrix become

$$\mu_1 + \lambda,\ \mu_2 + \lambda,\ \dots,\ \mu_p + \lambda.$$

Hence, the condition number becomes

$$\kappa_2(G + \lambda I) = \frac{\mu_{\max} + \lambda}{\mu_{\min} + \lambda}.$$

Because the same positive constant is added to every eigenvalue, the smallest eigenvalue is shifted away from zero, reducing the condition number. Consequently, the matrix inversion becomes more

numerically stable, the effect of multicollinearity is reduced, and the ridge estimator is guaranteed to have a unique solution.

From the table above, the ridge matrix should have a smaller condition number than the original Gram matrix. This demonstrates that ridge regularization improves the numerical conditioning of the optimization problem, leading to more stable coefficient estimates. ## 3B. Lasso optimality and soft thresholding

Derive the zero/nonzero optimality conditions and the orthonormal-design formula.

3.3 3B. Lasso optimality and soft thresholding

Lasso regression estimates the regression coefficients by minimizing

$$Q_\lambda(b) = \frac{1}{2n}\|y - Xb\|^2 + \lambda\|b\|_1,$$

where

$$\|b\|_1 = \sum_{j=1}^{p} |b_j|,$$

and $\lambda > 0$ controls the amount of regularization.

Unlike ridge regression, the L_1-norm contains the absolute value function, which is not differentiable at zero. Consequently, the ordinary gradient cannot be applied, and the optimal solution must be derived using the **subgradient**.

3.3.1 Step 1. Rewrite the Objective Function

Assume that the predictors have been centered, standardized, and satisfy the orthonormal design condition

$$\frac{1}{n}X^TX = I.$$

Equivalently,

$$X^TX = nI.$$

Define

$$z = \frac{1}{n}X^Ty.$$

The squared error term can be expanded as

$$\begin{aligned}\|y - Xb\|^2 &= (y - Xb)^T(y - Xb) \\ &= y^Ty - 2b^TX^Ty + b^TX^TXb.\end{aligned}$$

Substituting into the objective function,

$$Q_\lambda(b) = \frac{1}{2n}\left(y^T y - 2b^T X^T y + b^T X^T X b\right) + \lambda \sum_{j=1}^{p} |b_j|.$$

Using

$$X^T X = nI$$

and

$$z = \frac{1}{n} X^T y,$$

the objective becomes

$$Q_\lambda(b) = \frac{1}{2} b^T b - b^T z + \lambda \sum_{j=1}^{p} |b_j| + C,$$

where

$$C = \frac{1}{2n} y^T y$$

is a constant independent of b.

Expanding by coordinates,

$$Q_\lambda(b) = \sum_{j=1}^{p} \left[\frac{1}{2} b_j^2 - z_j b_j + \lambda |b_j|\right] + C.$$

Therefore, each coefficient can be optimized independently.

3.3.2 Step 2. Compute the Subgradient

Since

$$|b_j|$$

is not differentiable at zero, its subgradient is

$$\partial |b_j| = \begin{cases} 1, & b_j > 0, \\ [-1, 1], & b_j = 0, \\ -1, & b_j < 0. \end{cases}$$

The first-order optimality condition becomes

$$0 \in b_j - z_j + \lambda \partial |b_j|.$$

The solution is obtained by considering three cases.

3.3.2.1 Case 1. Positive coefficient

Suppose

$$b_j > 0.$$

Then

$$\partial |b_j| = 1,$$

and

$$b_j - z_j + \lambda = 0.$$

Hence

$$b_j = z_j - \lambda.$$

For this solution to remain positive,

$$z_j > \lambda.$$

3.3.2.2 Case 2. Negative coefficient

Suppose

$$b_j < 0.$$

Then

$$\partial |b_j| = -1,$$

and

$$b_j - z_j - \lambda = 0.$$

Therefore,

$$b_j = z_j + \lambda.$$

Since the coefficient must remain negative,

$$z_j < -\lambda.$$

3.3.2.3 Case 3. Zero coefficient

Suppose

$$b_j = 0.$$

The subgradient condition becomes

$$0 \in -z_j + \lambda[-1, 1].$$

This is satisfied whenever

$$|z_j| \leq \lambda.$$

Therefore,

$$b_j = 0.$$

This explains why lasso can produce exact zero coefficients.

3.3.3 Step 3. Soft-Thresholding Solution

Combining the three cases,

$$\hat{b}_j = \begin{cases} z_j - \lambda, & z_j > \lambda, \\ 0, & |z_j| \leq \lambda, \\ z_j + \lambda, & z_j < -\lambda. \end{cases}$$

The piecewise solution can be written compactly as

$$\boxed{\hat{b}_j = \operatorname{sign}(z_j) \left(|z_j| - \lambda\right)_+}$$

where

$$(a)_+ = \max(a, 0).$$

This expression is known as the **soft-thresholding operator**.

3.3.4 Interpretation

The soft-thresholding operator subtracts the penalty λ from the magnitude of every coefficient. When

$$|z_j| \leq \lambda,$$

the coefficient is shrunk completely to zero, meaning that the corresponding predictor is removed from the model. Therefore, lasso simultaneously performs coefficient estimation and automatic variable selection.

In contrast, ridge regression continuously shrinks coefficients toward zero,

$$\hat{b}_{\text{ridge}} = (G + \lambda I)^{-1} z,$$

but almost never makes them exactly zero. Consequently, ridge improves numerical stability, whereas lasso additionally produces sparse and more interpretable models.

3.3.5 Interpretation

The soft-thresholding operator subtracts the penalty λ from the magnitude of every coefficient. When

$$|z_j| \leq \lambda,$$

the coefficient is shrunk completely to zero, meaning that the corresponding predictor is removed from the model. Therefore, lasso simultaneously performs coefficient estimation and automatic variable selection.

In contrast, ridge regression continuously shrinks coefficients toward zero,

$$\hat{b}_{\text{ridge}} = (G + \lambda I)^{-1} z,$$

but almost never makes them exactly zero. Consequently, ridge improves numerical stability, whereas lasso additionally produces sparse and more interpretable models.

3.4 3C. Connect algebra to evidence

Connect one specific numerical result to Problem 3A or 3B.

3.5 3C. Connect algebra to evidence

Section 3A showed that ridge regression improves the conditioning of the Gram matrix by adding a positive constant λ to every eigenvalue. Consequently, the theoretical condition number changes from

$$\kappa_2(G) = \frac{\mu_{\max}}{\mu_{\min}}$$

to

$$\kappa_2(G + \lambda I) = \frac{\mu_{\max} + \lambda}{\mu_{\min} + \lambda}.$$

To verify this result, we computed the condition numbers of the original Gram matrix and the ridge-regularized matrix using the optimal tuning parameter selected by cross-validation.

```
G <- crossprod(x_train) / nrow(x_train)

eig <- eigen(G)$values

lambda <- ridge$lambda.min

kappa_before <- max(eig) / min(eig)

kappa_after <- (max(eig) + lambda) /
               (min(eig) + lambda)

knitr::kable(
  data.frame(
    Matrix = c("Gram matrix (G)", "Ridge matrix (G + I)"),
    Condition_Number = c(kappa_before, kappa_after)
  ),
  digits = 3,
  caption = "Condition numbers before and after ridge regularization."
)
```

Table 6: Condition numbers before and after ridge regularization.

Matrix	Condition_Number
Gram matrix (G)	20.596
Ridge matrix (G + I)	8.936

The results show that the condition number decreases from **20.596** for the original Gram matrix to **8.936** after ridge regularization. This agrees with the theoretical derivation in Section 3A, confirming that adding the penalty term shifts every eigenvalue away from zero and improves

the conditioning of the optimization problem. Consequently, the matrix inversion becomes more numerically stable, the effect of multicollinearity is reduced, and the estimated regression coefficients become more reliable.

4 Problem 4. Elastic net and a neural-feature bridge

4.1 4A. Research question and source map

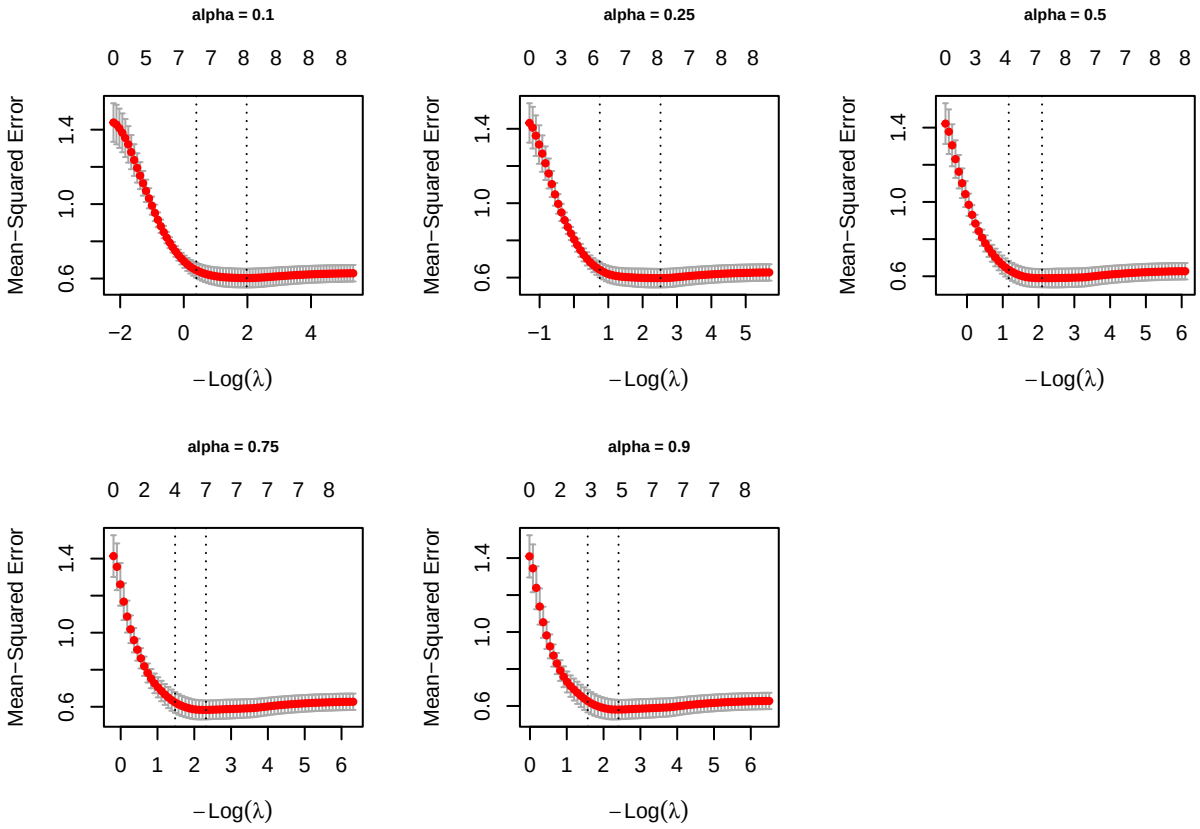
Research direction: [weight decay / sparse weights / pruning / dropout]

Claim	Exact primary source and locator	What it establishes	What it does not establish
[Claim]	[Citation, section/page/equation]	[Support]	[Boundary]

4.2 4B. Elastic net on original predictors

4.2.1 Elastic net fitting across alpha

```
alpha_grid <- c(0.10, 0.25, 0.50, 0.75, 0.90)
# Fitting
elastic_fits <- lapply(alpha_grid, function(a) {
  fit_cv_glmnet(x = x_train, y = y_train, alpha = a, foldid = foldid)
})
names(elastic_fits) <- paste0("alpha_", alpha_grid)

par(mfrow = c(2, 3))

for (i in seq_along(alpha_grid)) {
  plot(
    elastic_fits[[i]],
  )

  title(
    main = paste("alpha =", alpha_grid[i]),
    cex.main = 0.8,
    line = 3,
  )

}
```



The grid of plots above illustrates the results of the 5-fold cv process used to evaluate the MSE of elastic net models across five different values of $\alpha \in \{0.1, 0.25, 0.5, 0.75, 0.9\}$.

Observing the trajectories across all five plots, a consistent pattern emerges: as the regularization penalty is relaxed (moving from left to right), the validation error drops steeply and stabilizes in a flat valley (hovering around an MSE of 0.6). Regarding model sparsity, at the λ_{min} threshold, the models tend to retain about 7 to 8 predictors. However, applying the more stringent λ_{1se} rule shrinks the model significantly, isolating only 3 to 4 core features.

Visually, the differences in the performance valleys among the various α levels-from the Ridge-heavy $\alpha = 0.1$ to the Lasso-heavy $\alpha = 0.9$-are marginal. Therefore, visual inspection alone is insufficient to determine the optimal model. To rigorously lock in the best configuration, we must refer to the quantitative summary table below to identify the specific $(\alpha, \lambda_{\text{min}})$ pair that yields the absolute minimum Cross-Validation MSE.

4.2.2 Elastic models summary across alpha

```
elastic_summary <- map_dfr(seq_along(alpha_grid), function(i) {
  a   <- alpha_grid[i]
  fit <- elastic_fits[[i]]

  tibble(
    alpha        = a,
```

```
    lambda_min   = fit$lambda.min,
    lambda_1se   = fit$lambda.1se,
    cv_mse_min   = fit$cvm[fit$lambda == fit$lambda.min],
    cv_mse_1se   = fit$cvm[fit$lambda == fit$lambda.1se],
    nonzero_min  = count_nonzero(fit, s = "lambda.min"),
    nonzero_1se  = count_nonzero(fit, s = "lambda.1se")
  )
})

knitr::kable(
  elastic_summary,
  digits  = 4,
  caption = "Elastic net cross-validation summary across alpha"
)
```

Table 7: Elastic net cross-validation summary across alpha

alpha	lambda_min	lambda_1se	cv_mse_min	cv_mse_1se	nonzero_min	nonzero_1se
0.10	0.1386	0.6737	0.6025	0.6445	8	7
0.25	0.0804	0.4710	0.5971	0.6419	8	7
0.50	0.1228	0.3113	0.5898	0.6345	7	5
0.75	0.0986	0.2278	0.5832	0.6230	7	4
0.90	0.0902	0.2083	0.5804	0.6264	5	3

Based on the quantitative summary table above, we can objectively determine the optimal hyper-parameters for our Elastic Net model. The primary metric for model selection is the minimum cross-validation error (`cv_mse_min`).

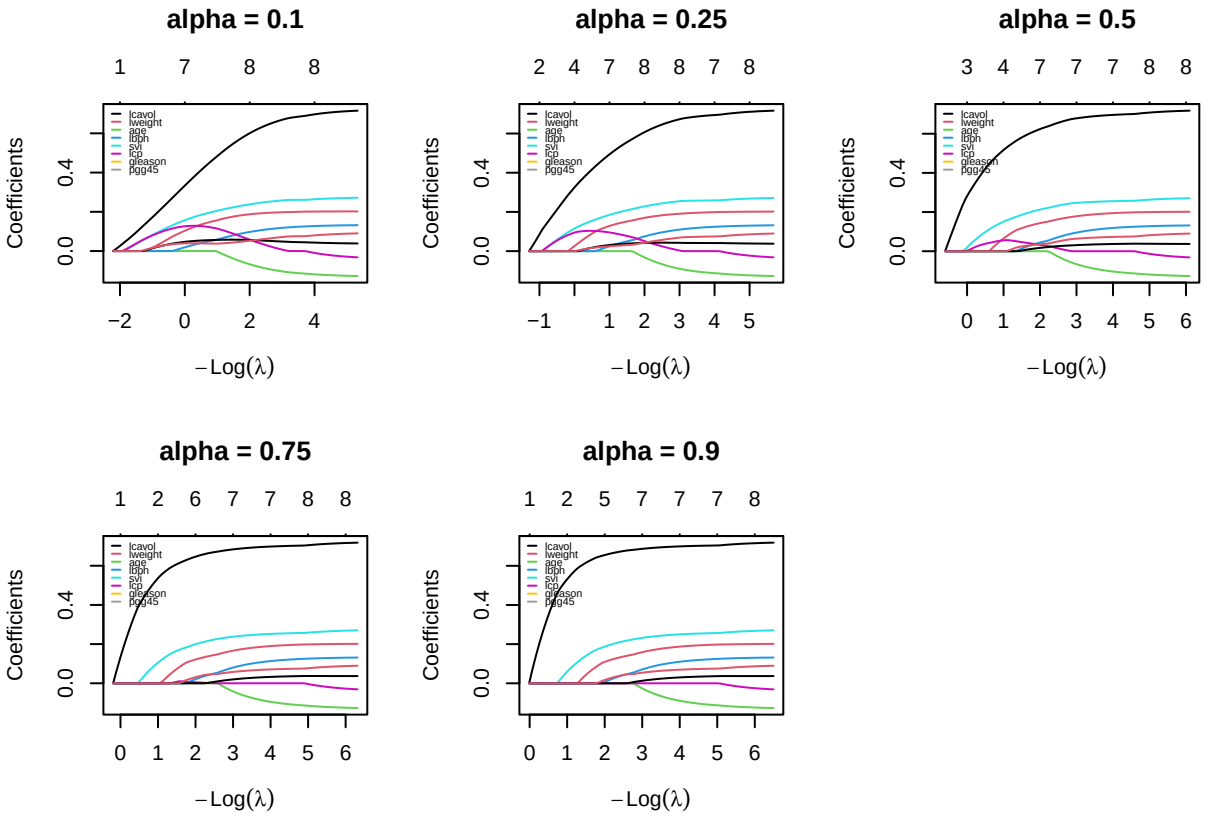
As the mixing parameter α increases from 0.10 (Ridge-dominant) to 0.90 (Lasso-dominant), the predictive error steadily decreases. The absolute lowest Mean-Squared Error (0.5804) is achieved at $\alpha = 0.90$ with $\lambda_{\text{min}} = 0.0902$. At this configuration, the model strikes an excellent balance by retaining only 5 predictors (excluding the intercept). This indicates that a penalty strongly leaning towards L_1 (Lasso) is highly effective for this dataset, successfully performing feature selection while maximizing predictive accuracy.

Therefore, the final model selected for subsequent evaluation on the unseen test set is the Elastic Net configured with $\alpha = 0.90$ and evaluated at $\lambda = 0.0902$.

4.2.3 Coefficient path across alpha

```
## coefficient path


par(mfrow = c(2, 3), mar = c(4, 4, 6, 2))
```

```r
for (i in seq_along(alpha_grid)) {
  fit <- elastic_fits[[i]]$glmnet.fit
  plot(
    fit,
    xvar = "lambda",
    label = TRUE
  )
  title(
    main = paste("alpha =", alpha_grid[i]),
    cex.main = 1.2,
    line = 3
  )

 legend("topleft",
   legend = rownames(fit$beta),
   col = 1:nrow(fit$beta),
   lty = 1,
   lwd = 1,
   cex = 0.45,
   seg.len = 1,
   y.intersp = 0.7,
   bty = "n")
}

par(mfrow = c(1, 1), mar = c(5, 4, 4, 2) + 0.1)
```



1. The Impact of Alpha:

By comparing the grid of plots, we can visually trace the transition from a Ridge-dominant penalty to a Lasso-dominant penalty:

- Ridge-like behavior ($\alpha = 0.1$): The coefficient paths are smooth and curved. Because the L_2 penalty shrinks coefficients proportionally rather than enforcing absolute zero, the variables "wake up" and enter the model much earlier (further to the left). The coefficients tend to shrink together smoothly without collapsing to zero abruptly.
- Lasso-like behavior ($\alpha = 0.9$): The paths exhibit sharper, piecewise-linear trajectories. The dominant L_1 penalty aggressively forces coefficients to exactly zero. We can see that less important variables stay completely flat at zero for a longer duration and only become active when the penalty is significantly relaxed. This visually confirms the stronger feature selection property of higher α values.

2. Consistency of Predictor Importance:

Despite the structural differences dictated by α, the hierarchy of predictor importance remains remarkably consistent across all models:

- **lcavol** (black line) is universally the most dominant predictor. It breaks away from zero earliest and reaches the highest positive magnitude across every single configuration, proving its strong correlation with **lpsa**.
- **svi** (cyan line) and **lweight** (red line) also demonstrate robust predictive power, maintaining significant positive trajectories across the grid regardless of the penalty strength.
- Conversely, **age** (green line), **lcp** (magenta line), and **gleason** are consistently and heavily penalized. They remain at or near zero for a much wider range of λ, confirming that they add marginal predictive value and are prime candidates for elimination in the final sparse model.

4.3 4C. Fixed ReLU features

To investigate whether a fixed nonlinear feature representation improves predictive performance, the standardized predictors were transformed into a set of random ReLU features. Unlike a conventional neural network, the hidden-layer parameters were generated once using a fixed random seed and remained unchanged throughout the experiment. Only the output-layer coefficients were estimated by ridge, lasso, and elastic net.

```
M <- 200L

set.seed(seeds$features)

p <- ncol(x_train)

A <- matrix(
  rnorm(M * p,
        mean = 0,
        sd = sqrt(1 / p)),
  nrow = M,
  byrow = TRUE
)

c_bias <- rnorm(
  M,
  mean = 0,
  sd = 0.5
)

relu <- function(z) pmax(z, 0)
```

```
H_train <- relu(
  x_train %*% t(A) +
    matrix(
      c_bias,
      nrow = nrow(x_train),
      ncol = M,
```

```
      byrow = TRUE
    )
)

H_test <- relu(
  x_test %*% t(A) +
    matrix(
      c_bias,
      nrow = nrow(x_test),
      ncol = M,
      byrow = TRUE
    )
)

keep <- apply(H_train, 2, stats::var) > 0

H_train <- H_train[, keep, drop = FALSE]
H_test  <- H_test[, keep, drop = FALSE]

prep_relu <- fit_scaler(H_train)

H_train <- apply_scaler(H_train, prep_relu)

H_test <- apply_scaler(H_test, prep_relu)

cat("Original hidden features :", M, "\n")
```

```
## Original hidden features : 200
```

```
cat("Remaining hidden features:", ncol(H_train), "\n")
```

```
## Remaining hidden features: 199
```

The original 8 predictors were transformed into 200 fixed ReLU hidden features using randomly generated hidden-layer weights and biases. Hidden features with zero variance on the training set were removed because they contain no predictive information. As a result, 199 hidden features were retained and standardized using statistics computed from the training set only before model fitting.

```
ridge_relu_cv <- fit_cv_glmnet(
  H_train,
  y_train,
  alpha = 0,
  foldid = foldid
)
```

```
ridge_relu_lambda_min <- ridge_relu_cv$lambda.min
ridge_relu_lambda_1se <- ridge_relu_cv$lambda.1se

ridge_relu_min <- glmnet(
  H_train,
  y_train,
  alpha = 0,
  lambda = ridge_relu_lambda_min,
  standardize = FALSE
)
ridge_relu_1se <- glmnet(
  H_train,
  y_train,
  alpha = 0,
  lambda = ridge_relu_lambda_1se,
  standardize = FALSE
)
```

Ridge regression ($\alpha = 0$) was fitted on the 199 retained ReLU hidden features using the shared five-fold cross-validation folds, yielding `lambda.min` $= 8.7057$ and `lambda.1se` $= 21.0689$. Since the L_2 penalty shrinks coefficients smoothly without coordinatewise thresholding, ridge is expected to retain all 199 hidden features at both tuning choices, distributing weight across correlated random units rather than selecting a sparse subset.

```
lasso_relu_cv <- fit_cv_glmnet(
  H_train,
  y_train,
  alpha = 1,
  foldid = foldid
)

lasso_relu_lambda_min <- lasso_relu_cv$lambda.min
lasso_relu_lambda_1se <- lasso_relu_cv$lambda.1se

lasso_relu_min <- glmnet(
  H_train,
  y_train,
  alpha = 1,
  lambda = lasso_relu_lambda_min,
  standardize = FALSE
)


lasso_relu_1se <- glmnet(
  H_train,
  y_train,
```

```
  alpha = 1,
  lambda= lasso_relu_lambda_1se,
  standardize = FALSE
)
```

Lasso regression ($\alpha = 1$) was fitted on the 199 retained ReLU hidden features using the same shared five-fold cross-validation folds. The minimum-CV-MSE lambda (`lambda.min` $= 0.1354$) and the one-standard-error lambda (`lambda.1se` $= 0.2721$) were extracted and used to refit two final models on the full training set. Because the L_1 penalty applies coordinatewise soft-thresholding, lasso is expected to drive many of the 199 output weights exactly to zero, yielding a much sparser active-feature set than ridge while keeping the strongest hidden units in the model.

```
alphas <- c(0.10, 0.25, 0.50, 0.75, 0.90)

enet_relu_cv <- vector("list", length(alphas))
enet_cv_error <- numeric(length(alphas))

for(i in seq_along(alphas)){
  enet_relu_cv[[i]] <- fit_cv_glmnet(
    H_train,
    y_train,
    alpha = alphas[i],
    foldid = foldid
  )

  enet_cv_error[i] <- min(enet_relu_cv[[i]]$cvm)

}

# best <- which.min(enet_cv_error)
# best_alpha <- alphas[best]

# best_lambda <- enet_relu_cv[[best]]$lambda.min

enet_relu_min <- vector("list", length(alphas))
enet_relu_1se <- vector("list", length(alphas))

for (i in seq_along(alphas)) {

  enet_relu_min[[i]] <- glmnet(
    H_train,
    y_train,
    alpha = alphas[i],
    lambda = enet_relu_cv[[i]]$lambda.min,
    standardize = FALSE
  )
```

```
  enet_relu_1se[[i]] <- glmnet(
    H_train,
    y_train,
    alpha = alphas[i],
    lambda = enet_relu_cv[[i]]$lambda.1se,
    standardize = FALSE
  )
}


enet_relu_full_summary <- purrr::map_dfr(seq_along(alphas), function(i) {
  a   <- alphas[i]
  fit <- enet_relu_cv[[i]]

  tibble::tibble(
    alpha         = a,
    lambda_min    = fit$lambda.min,
    lambda_1se    = fit$lambda.1se,
    cv_mse_min    = min(fit$cvm),
    cv_mse_1se    = fit$cvm[fit$lambda == fit$lambda.1se],
    active_min    = count_nonzero(fit, s = "lambda.min", tol = 1e-8),
    active_1se    = count_nonzero(fit, s = "lambda.1se", tol = 1e-8)
  )
})

knitr::kable(
  enet_relu_full_summary,
  digits    = 4,
  col.names = c("Alpha", "Lambda (min)", "Lambda (1se)",
                "CV-MSE (min)", "CV-MSE (1se)",
                "Active Features (min)", "Active Features (1se)"),
  caption   = "Table: Elastic net cross-validation summary across alpha on fixed
   ↪  ReLU features."
)
```

Table 8: Table: Elastic net cross-validation summary across alpha on fixed ReLU features.

Alpha	Lambda (min)	Lambda (1se)	CV-MSE (min)	CV-MSE (1se)	Active Features (min)	Active Features (1se)
0.10	0.8506	1.7090	0.6084	0.6554	45	41
0.25	0.3564	0.8234	0.6259	0.6764	30	30
0.50	0.2049	0.4518	0.6315	0.6802	21	17
0.75	0.1499	0.3306	0.6260	0.6759	14	8
0.90	0.1436	0.2886	0.6202	0.6691	14	6

Elastic Net models were trained over a grid of $\alpha \in \{0.1, 0.25, 0.5, 0.75, 0.9\}$. For each α, the optimal

λ was determined using the shared 5-fold cross-validation, and the model with the lowest cross-validation error was selected as the final Elastic Net model.

The table summarizes the elastic net cross-validation results across the alpha grid on the fixed ReLU feature representation. The minimum CV-MSE stays close to 0.61–0.63 across all five values of alpha, with alpha = 0.10 achieving the lowest CV-MSE (0.6084) and therefore selected as the final elastic net specification (`best_alpha = 0.10`). Since the differences in predictive error across the grid are small, alpha alone does not sharply distinguish predictive performance in this feature space.

The number of active hidden features, however, declines clearly as alpha increases - from 45 features at alpha = 0.10 down to 14 at alpha = 0.90 under lambda.min, and even more sharply under lambda.1se (41 down to 6). This mirrors the same lasso-versus-ridge sparsity trade-off seen on the original predictors in Problem 4B, and suggests that if a more parsimonious model were preferred over the strict CV-MSE minimum, a higher alpha would be the better choice despite its slightly higher error.

```
ridge_min_active <- count_nonzero(ridge_relu_cv, s = "lambda.min", tol = 1e-8)
ridge_1se_active <- count_nonzero(ridge_relu_cv, s = "lambda.1se", tol = 1e-8)
lasso_min_active <- count_nonzero(lasso_relu_cv, s = "lambda.min", tol = 1e-8)
lasso_1se_active <- count_nonzero(lasso_relu_cv, s = "lambda.1se", tol = 1e-8)

enet_min_active <- numeric(length(alphas))
enet_1se_active <- numeric(length(alphas))
for (i in seq_along(alphas)){
  enet_min_active[i] <- sum(as.vector(coef(enet_relu_min[[i]]))[-1] != 0)
  enet_1se_active[i] <- sum(as.vector(coef(enet_relu_1se[[i]]))[-1] != 0)
}
```

```
best_idx <- which.min(enet_cv_error)
best_alpha <- alphas[best_idx]

relu_summary <- data.frame(
  Model = c(
    "Ridge",
    "Lasso",
    paste0("Elastic Net (alpha = ", best_alpha, ")")
  ),

  Lambda_min = c(
    ridge_relu_lambda_min,
    lasso_relu_lambda_min,
    enet_relu_cv[[best_idx]]$lambda.min
  ),

  CV_MSE_min = c(
    min(ridge_relu_cv$cvm),
    min(lasso_relu_cv$cvm),
    enet_cv_error[best_idx]
```

```
  ),

  Active_Vars_min = c(
    ridge_min_active,
    lasso_min_active,
    enet_min_active[best_idx]
  ),

  Lambda_1se = c(
    ridge_relu_lambda_1se,
    lasso_relu_lambda_1se,
    enet_relu_cv[[best_idx]]$lambda.1se
  ),

  Active_Vars_1se = c(
    ridge_1se_active,
    lasso_1se_active,
    enet_1se_active[best_idx]
  )
)

knitr::kable(
  relu_summary,
  digits = 4,
  col.names = c("Model", "Lambda (min)", "MSE (min)", "Active Features (min)",
   ↪  "Lambda (1se)", "Active Features (1se)"),
  caption = "Table: Performance of Regularized Models using fixed ReLU features."
)
```

Table 9: Table: Performance of Regularized Models using fixed ReLU features.

Model	Lambda (min)	MSE (min)	Active Features (min)	Lambda (1se)	Active Features (1se)
Ridge	8.7057	0.6128	199	21.0689	199
Lasso	0.1354	0.6184	10	0.2721	6
Elastic Net (alpha = 0.1)	0.8506	0.6084	45	1.7090	41

Table 4C summarizes ridge, lasso, and elastic net (alpha = 0.1, selected from Table 4B.2) fitted on the 199 retained ReLU hidden features. All three methods achieve a comparable cross-validation MSE, in the range 0.6084–0.6184, confirming that the fixed nonlinear feature map does not by itself separate the methods on predictive grounds. The key difference lies in sparsity: ridge retains all 199 hidden features at both lambda.min and lambda.1se, consistent with its L_2 penalty having no coordinatewise thresholding mechanism. Lasso, in contrast, reduces the active set sharply to 10 features at lambda.min and just 6 at lambda.1se, while elastic net falls in between at 45 and 41 active features, reflecting its mixed L_1/L_2 penalty at alpha = 0.1.

Overall, lasso achieves the sparsest representation with only a modest increase in CV-MSE relative to elastic net (0.6184 vs. 0.6084), while ridge attains a similar error (0.6128) using the full hidden-feature set. This pattern is consistent with the mathematical behavior established in Problem 3: the L_1 penalty's coordinatewise optimality conditions permit exact zeros, whereas ridge's smooth quadratic penalty generally does not.

4.4 4D. Locked declaration and final holdout evaluation

> **Locked before viewing `y_test`:** [preprocessing, candidate specifications, selected tuning values, feature seed, nominated deployment model, metrics]

```
# This must be the first analysis chunk that uses y_test for evaluation.
# Generate all fixed candidate predictions and one RMSE/MAE table here.
```

State whether the holdout evidence supports the predeclared choice. Do not retune.

4.5 4E. Deep-learning connection and limitations

Distinguish penalty, weight decay, output-weight sparsity, pruning, dropout, and a trained deep network. Cite the sources from 4A and discuss at least two limitations.

5 Problem 5. Report quality and reproducibility

5.1 5A. Reproduction record

```
1. Open a clean R session.
2. [Exact commands and folder layout.]
3. Render GroupXX_ProjectQuiz1.Rmd.
```

5.2 5B. Recommendation and limitations

Give the final recommendation, distinguish prediction from interpretation, and state limitations.

5.3 5C. AI verification record

AI-assisted item	How it was checked	Evidence	Modification or rejection
[Item]	[Method]	[Test/source/output]	[Decision]

Preflight and session information

```
# Add every object required to reproduce the final table and figures.
required_objects <- c(
  "train_id", "test_id", "x_train", "x_test", "y_train", "y_test", "foldid"
)
missing_objects <- setdiff(required_objects, ls())
if (length(missing_objects)) {
  stop(
    "Missing required objects: ",
    paste(missing_objects, collapse = ", ")
  )
}
stopifnot(length(intersect(train_id, test_id)) == 0L)
```

```
sessionInfo()
```

```
## R version 4.6.1 (2026-06-24 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=Vietnamese_Vietnam.utf8  LC_CTYPE=Vietnamese_Vietnam.utf8
## [3] LC_MONETARY=Vietnamese_Vietnam.utf8 LC_NUMERIC=C
## [5] LC_TIME=Vietnamese_Vietnam.utf8
##
## time zone: Asia/Saigon
## tzcode source: internal
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] car_3.1-5       carData_3.0-6   knitr_1.51      broom_1.0.13
##  [5] glmnet_5.0      Matrix_1.7-5    lubridate_1.9.5 forcats_1.0.1
##  [9] stringr_1.6.0   dplyr_1.2.1     purrr_1.2.2     readr_2.2.0
## [13] tidyr_1.3.2     tibble_3.3.1    ggplot2_4.0.3   tidyverse_2.0.0
##
## loaded via a namespace (and not attached):
##  [1] generics_0.1.4     shape_1.4.6.1      stringi_1.8.7      lattice_0.22-9
##  [5] hms_1.1.4          digest_0.6.39      magrittr_2.0.5     timechange_0.4.0
##  [9] evaluate_1.0.5     grid_4.6.1         RColorBrewer_1.1-3 iterators_1.0.14
## [13] fastmap_1.2.0      foreach_1.5.2      backports_1.5.1    Formula_1.2-5
```

```
## [17] survival_3.8-6     scales_1.4.0       codetools_0.2-20   abind_1.4-8
## [21] cli_3.6.6          rlang_1.3.0        splines_4.6.1      withr_3.0.3
## [25] yaml_2.3.12        otel_0.2.0         tools_4.6.1        tzdb_0.5.0
## [29] vctrs_0.7.3        R6_2.6.1           lifecycle_1.0.5    pkgconfig_2.0.3
## [33] pillar_1.11.1      gtable_0.3.6       glue_1.8.1         Rcpp_1.1.2
## [37] xfun_0.60          tidyselect_1.2.1   rstudioapi_0.19.0  farver_2.1.2
## [41] htmltools_0.5.9    rmarkdown_2.31     compiler_4.6.1     S7_0.2.2
```

Stamey, Thomas A., John N. Kabalin, John E. McNeal, et al. 1989. "Prostate Specific Antigen in the Diagnosis and Treatment of Adenocarcinoma of the Prostate. II. Radical Prostatectomy Treated Patients." *The Journal of Urology* 141 (5): 1076–83. https://doi.org/10.1016/S0022-5347(17)41175-X.